



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**título del TFG
Documentación Técnica**



Presentado por nombre alumno
en Universidad de Burgos — 30 de junio
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	10
Apéndice B Especificación de Requisitos	15
B.1. Introducción	15
B.2. Objetivos generales	15
B.3. Catálogo de requisitos	16
B.4. Especificación de requisitos	21
Apéndice C Especificación de diseño	29
C.1. Introducción	29
C.2. Diseño de datos	29
C.3. Diseño procedimental	32
C.4. Diseño arquitectónico	39
Apéndice D Documentación técnica de programación	49
D.1. Introducción	49
D.2. Estructura de directorios	49
D.3. Manual del programador	52

D.4. Compilación, instalación y ejecución del proyecto	57
D.5. Pruebas del sistema	57
Apéndice E Documentación de usuario	61
E.1. Introducción	61
E.2. Requisitos de usuarios	61
E.3. Instalación	61
E.4. Manual del usuario	61
Apéndice F Anexo de sostenibilización curricular	63
F.1. Introducción a la Sostenibilidad	63
F.2. Dimensión Medioambiental	63
F.3. Dimensión Económica	64
F.4. Dimensión Social y Ética Profesional	64
F.5. Alineación con los Objetivos de Desarrollo Sostenible (ODS)	65
Bibliografía	67

Índice de figuras

A.1. Gráfico Burndown del Sprint 1	3
A.2. Gráfico Burndown del Sprint 2	4
A.3. Gráfico Burndown del Sprint 3	5
A.4. Gráfico Burndown del Sprint 4	6
A.5. Gráfico Burndown del Sprint 5	7
A.6. Gráfico Burndown del Sprint 6	8
A.7. Gráfico Burndown del Sprint 7	9
A.8. Gráfico Burndown del Sprint 8	10
C.1. Diagrama Entidad-Relación Lógico del Sistema	32
C.2. Diagrama de Secuencia: Flujo de Autenticación	35
C.3. Diagrama de Secuencia: Planificación de Cobertura Agrícola	36
C.4. Diagrama de Secuencia: Inicio de una Misión Agrícola	37
C.5. Diagrama de Secuencia: Ciclo de Telemetría Asíncrona	38
C.6. Diagrama de Secuencia: Flujo de Visualización Analítica	39
C.7. Diagrama de Arquitectura de Software: Interacción entre Cliente, Servidor y Base de Datos	40
C.8. Diagrama de Infraestructura y Despliegue: Contenedores Docker	41
C.9. Diagrama de Arquitectura Interna del Backend (Node.js y Motor Activo)	43
C.10. Diagrama de Arquitectura Interna del Frontend (Ecosistema React)	45
D.1. Diagrama de flujo del ciclo de vida de una petición HTTP en la API REST.	54
D.2. Diagrama del flujo asíncrono de datos emitidos por el simulador mediante WebSockets.	56

Índice de tablas

A.1. Resumen de licencias de las herramientas y tecnologías utilizadas	13
B.1. CU-1 Iniciar sesión.	22
B.2. CU-2 Gestionar cuentas de empleado.	23
B.3. CU-3 Control manual del robot.	24
B.4. CU-4 Diseñar y ejecutar misión autónoma.	25
B.5. CU-5 Retorno automático por cálculo de distancia.	26
B.6. CU-6 Análisis integral de datos agronómicos.	27

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Este anexo detalla la gestión y planificación del desarrollo de AgroSkopos, así como el análisis de su viabilidad técnica, económica y legal. Dada la naturaleza evolutiva del *software*, el proyecto se ha gestionado empleando la metodología ágil Scrum, adaptada a un entorno de desarrollo individual.

Para la gestión del ciclo de vida de la aplicación se ha utilizado la herramienta **Zube**, estrechamente integrada con GitHub. El proyecto se ha fragmentado teóricamente en iteraciones o *sprints* de dos semanas de duración. La estimación del esfuerzo de las tareas (Issues) se ha realizado midiendo las horas de trabajo estimadas, lo que resulta más representativo que los tradicionales *Story Points* en el contexto de un Trabajo de Fin de Grado compaginado con la actividad académica. El seguimiento del trabajo se documenta a través de gráficos de trabajo pendiente (*Burndown Charts*) generados automáticamente por la plataforma.

Adopción metodológica: Durante los primeros compases del proyecto (septiembre y octubre de 2025), el desarrollo se llevó a cabo sin una metodología formal, priorizando el autoaprendizaje tecnológico y la planificación general con el profesor tutor. Fue a mediados de noviembre cuando se tomó la decisión de adoptar formalmente Scrum con la herramienta **Zube**, con el propósito de imponer una estructura organizativa que obligara a cumplir plazos de “entrega” estrictos e incrementara la productividad frente a la elevada carga de trabajo.

Flexibilidad lectiva: La distribución temporal de los 9 *sprints* resultantes a lo largo del curso lectivo presenta cierta irregularidad y espacios

de inactividad entre las iteraciones. Esta flexibilidad en la planificación responde a la realidad de compatibilizar el desarrollo del *software* con la alta carga lectiva de la universidad (asistencia a clases, realización de trabajos y exámenes), así como con la dedicación a otros proyectos personales de programación. Asimismo, el inicio y cierre de los *sprints* ha estado condicionado por la disponibilidad conjunta del alumno y el tutor, ajustando las fechas para garantizar que las reuniones de validación y revisión del código se pudieran llevar a cabo.

A.2. Planificación temporal

El desarrollo del proyecto estructurado en iteraciones abarca desde mediados de noviembre hasta la fecha de entrega, distribuyendo el esfuerzo total en 9 Sprints. Adicionalmente, existió una fase previa de cimentación técnica. A continuación, se detallan los objetivos registrados para cada etapa y la respectiva gráfica de rendimiento de los Sprints.

Fase de Pre-desarrollo (Septiembre - Octubre 2025)

Desarrollo fundamental: Previo a la adopción de Scrum, se ejecutó una fase de desarrollo inicial (sin metodología formal ni control temporal por *Burndown*) orientada a asentar las bases estructurales del *frontend*. Durante estas semanas iniciales se diseñó y programó la cimentación de la Interfaz de Usuario (UI), incluyendo la pantalla principal interactiva y las vistas de *login*. A nivel arquitectónico, se implementó el enrutamiento base con React Router y la gestión de estado global a través de Zustand. Paralelamente, se sentaron las bases del visor cartográfico integrando la librería Leaflet, habilitando controles de capas, eventos de clic, un minimapa de orientación y modales informativos de estado de batería. Todas las funcionalidades de esta etapa operaron exclusivamente contra datos estáticos locales (*mocks*).

Sprint 1 (12 Nov. 2025 - 26 Nov. 2025)

Migración a entorno productivo: El objetivo primordial de esta iteración consistió en la transición hacia un entorno *backend* productivo, migrando la persistencia de datos desde un sistema de simulación estática hacia una arquitectura relacional sólida mediante la integración de PostgreSQL. A lo largo del *sprint*, el esfuerzo se diversificó en tres grandes bloques funcionales: en primer lugar, se consolidaron los mecanismos de seguridad y gestión de usuarios (autenticación JWT, cifrado con bcrypt, roles

de administrador y gestión de perfiles). En segundo lugar, se revolucionó la interfaz de usuario con mejoras de accesibilidad como el sistema de Modo Oscuro, un menú móvil completamente responsivo y un diseño avanzado de *dashboard* para el análisis de datos. Por último, se abordó el núcleo cartográfico y telemétrico, implementando comunicaciones en tiempo real para visualizar puntos de muestreo agronómico, habilitando herramientas de *geofencing* poligonal y desplegando un simulador interno capaz de generar y registrar datos telemétricos.

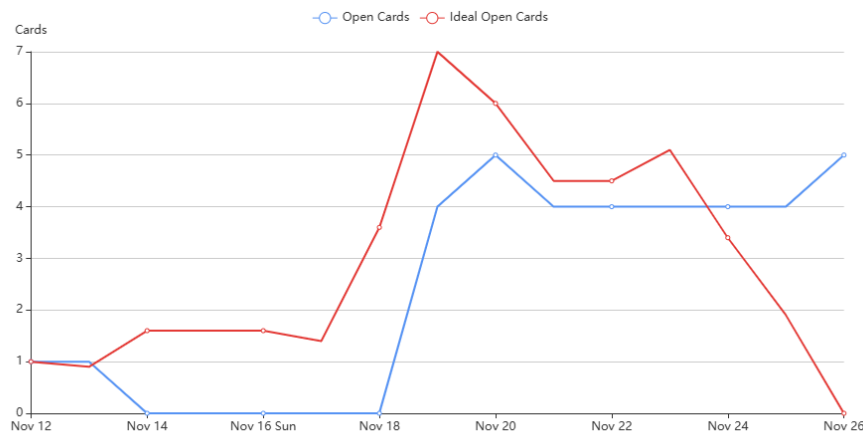


Figura A.1: Gráfico Burndown del Sprint 1

Sprint 2 (27 Dic. 2025 - 10 Ene. 2026)

Estabilización con Docker: Tras el periodo de pausa invernal, el esfuerzo se focalizó en la estabilización de los entornos de despliegue y desarrollo mediante la adopción de la tecnología de contenerización Docker. Este cambio estructural sentó las bases para un desarrollo mucho más robusto y escalable, minimizando las dependencias del entorno local. En paralelo a esta importante migración de infraestructura, se llevó a cabo un proceso iterativo de resolución de incidencias técnicas detectadas en fases anteriores y se dio continuidad a la implementación de funcionalidades operativas remanentes.

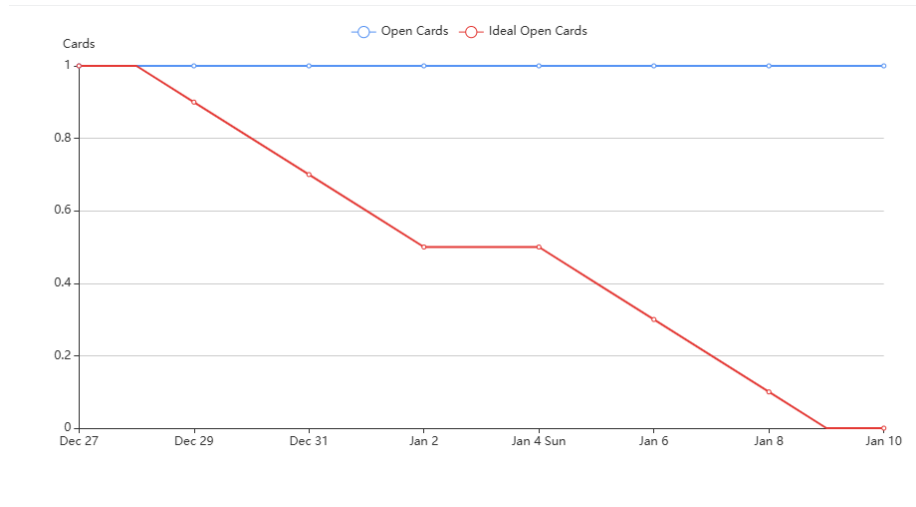


Figura A.2: Gráfico Burndown del Sprint 2

Sprint 3 (9 Feb. 2026 - 23 Feb. 2026)

Telemando y analítica: El núcleo de esta iteración fue la implementación de los módulos de telemando interactivo, habilitando el control bidireccional del vehículo. Esto incluyó no solo una interfaz para la navegación manual asistida, sino también sistemas de seguridad avanzados como una parada de emergencia (*E-Stop*), navegación automatizada multipunto y algoritmos para generar rutas de cobertura sistemática sobre parcelas agrícolas. Asimismo, se integró una interfaz de visualización de vídeo en tiempo real (*feed* de cámara) orientada a la monitorización. En el ámbito del análisis agronómico, se desarrollaron algoritmos espaciales para interpolar datos y representar la variabilidad de los parámetros recogidos a través de mapas de calor visuales e interactivos (*heatmaps*).

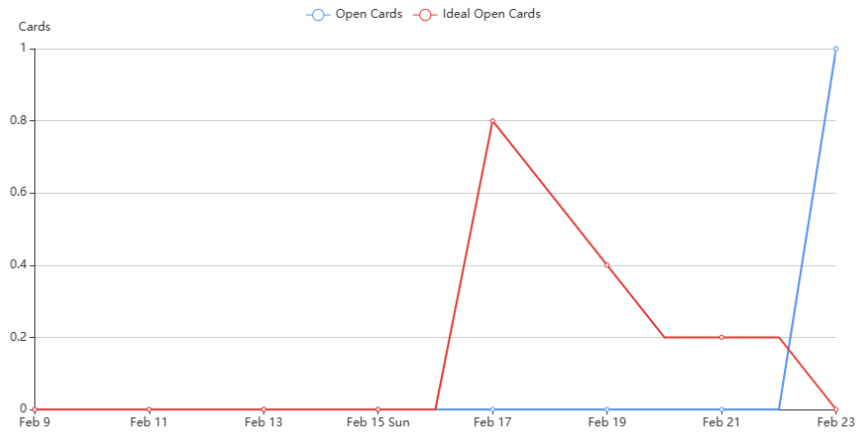


Figura A.3: Gráfico Burndown del Sprint 3

Sprint 4 (9 Mar. 2026 - 23 Mar. 2026)

Sistema de misiones e i18n: La iteración se estructuró en torno al desarrollo del sistema integral de Misiones, dotando al usuario de herramientas avanzadas para el diseño, almacenamiento y ejecución autónoma de rutas complejas sobre el terreno. Este módulo abarcó desde la configuración de la base de datos relacional y el panel de control de ejecución en tiempo real, hasta la generación de historiales y reportes analíticos posmisión. De forma concurrente, se reacondicionó la arquitectura de la interfaz gráfica integrando la librería *react-i18next*, lo que permitió establecer un selector dinámico para cambiar el idioma de la aplicación. Finalmente, se dedicó esfuerzo a refactorizar el código base para cumplir rigurosamente con los estándares de calidad de *SonarQube*.

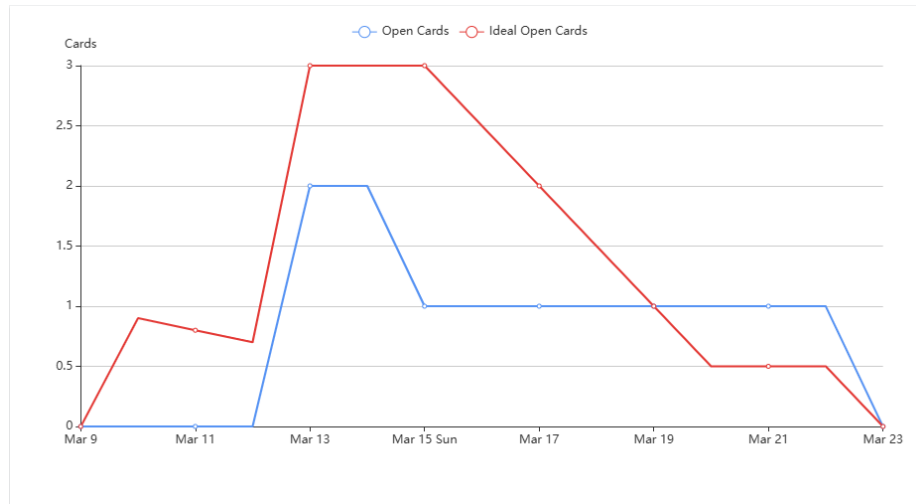


Figura A.4: Gráfico Burndown del Sprint 4

Sprint 5 (1 Abr. 2026 - 15 Abr. 2026)

Se acometió una expansión funcional transversal que afectó tanto a la capa de presentación como a la lógica de negocio subyacente. Entre los avances más destacados, se construyó la infraestructura base para la ingesta de telemetría dinámica y la monitorización de consumos energéticos, culminando con el diseño de una máquina de estados que otorga al robot la autonomía suficiente para retornar a la estación de recarga solar de forma automática. Asimismo, se perfeccionó la experiencia de usuario (UX) mediante gráficas de consumo temporal mejoradas, alertas visuales de estado y una barra lateral responsiva adaptada a dispositivos móviles. El Sprint también abordó tareas de arquitectura *backend*, mejorando la resiliencia de las sesiones de usuario e introduciendo un módulo de siembra de datos automáticos (*Seed*).

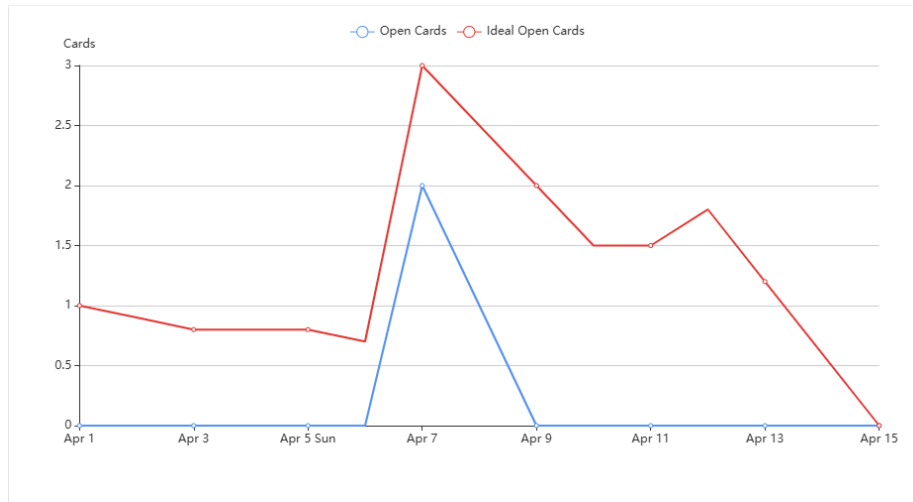


Figura A.5: Gráfico Burndown del Sprint 5

Sprint 6 (21 Abr. 2026 - 5 May. 2026)

A nivel de *software*, la iteración se centró en mejoras arquitectónicas significativas: se consolidó la reestructuración de todo el código bajo un patrón Modelo-Vista-Controlador (MVC) integrado dentro de un ecosistema Monorepositorio. Además, se culminó la transición del cliente hacia una Aplicación Web Progresiva (PWA), garantizando el funcionamiento fluido de la herramienta en entornos rurales sin conectividad estable, y se implementó un sistema de filtrado temporal avanzado para las gráficas telemétricas. Simultáneamente, y coincidiendo con este hito de madurez técnica del producto, se dio inicio formal a la redacción académica de la memoria del Trabajo de Fin de Grado.

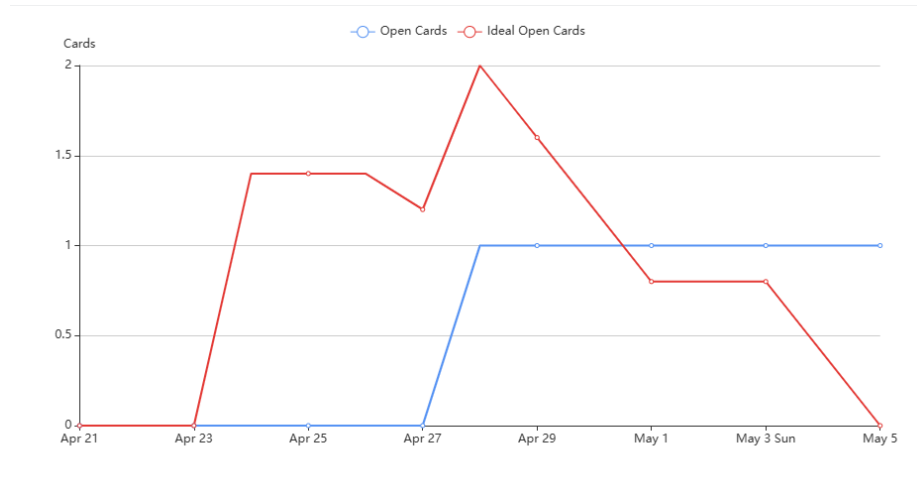


Figura A.6: Gráfico Burndown del Sprint 6

Sprint 7 (7 May. 2026 - 21 May. 2026)

Aseguramiento de Calidad (QA): Iteración dedicada prioritariamente a asegurar la robustez del código. Se desplegó una ambiciosa estrategia integral de pruebas automatizadas (*testing*) que cubrió tanto el cliente web (*React/Vite*) como el servidor (*Node.js/Express*). Para este último, se incorporaron herramientas como Jest y *Supertest*, orquestando pruebas de integración *End-to-End* (E2E) que validaron exhaustivamente los flujos principales de la API. A su vez, se enriqueció la experiencia de usuario resolviendo errores latentes, incorporando un sistema de avatares persistentes, habilitando la selección dinámica de parámetros agronómicos a registrar durante las misiones, y culminando la localización del sistema añadiendo soporte para el idioma portugués.

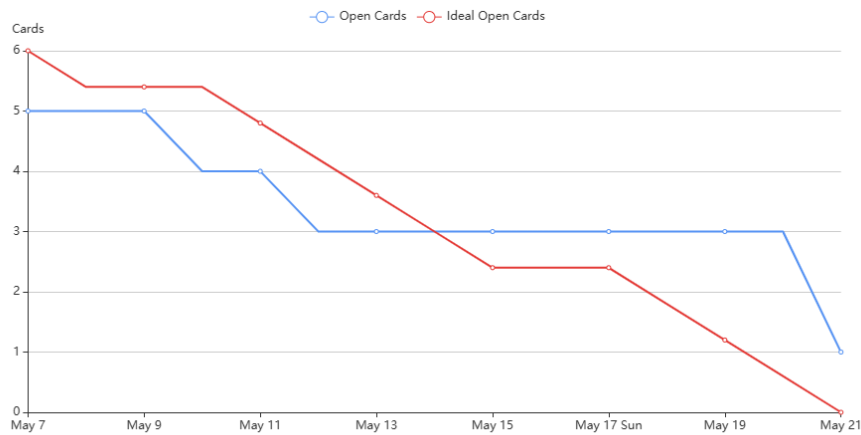


Figura A.7: Gráfico Burndown del Sprint 7

Sprint 8 (26 May. 2026 - 9 Jun. 2026)

El enfoque técnico se dividió entre el perfeccionamiento de la memoria académica y la consolidación definitiva del *backend*. La arquitectura lógica de la API REST experimentó una refactorización integral hacia un modelo estricto de tres capas, implementando soporte transaccional (ACID) para garantizar la consistencia en las operaciones SQL. La seguridad y estabilidad del sistema se maximizaron integrando validaciones modulares estrictas a través de esquemas *Zod* y centralizando el manejo de errores no capturados. Por último, se llevó a cabo un esfuerzo sustancial de estandarización documental, describiendo el código base de la lógica de cliente con *JSDoc* y generando una interfaz interactiva OpenAPI (Swagger) para exponer y testear los *endpoints*.

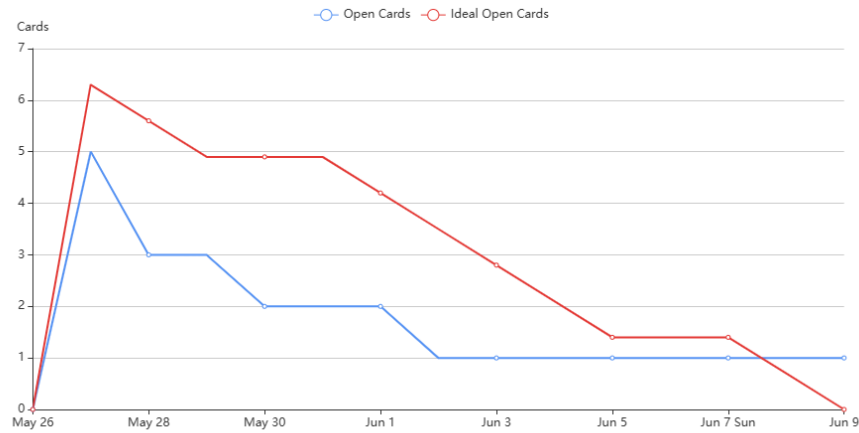


Figura A.8: Gráfico Burndown del Sprint 8

Sprint 9 (17 Jun. 2026 - 1 Jul. 2026)

Esta fase de cierre está dedicada de forma exclusiva a la maqueta y redacción final de la memoria principal, incluyendo el desarrollo exhaustivo de sus respectivos anexos técnicos y organizativos (A, B, C, D, E). El objetivo último de este ciclo es generar un documento académico sólido que fundamente y describa rigurosamente el trabajo de ingeniería desarrollado. (En curso, sin gráfico *Burndown*).

A.3. Estudio de viabilidad

Viabilidad económica

El presupuesto general del proyecto se divide en diferentes partidas de gasto.

Costes de Personal

Considerando las limitaciones temporales de un Trabajo de Fin de Grado, se asume una dedicación media de unas 20 horas semanales. Esta dedicación abarca la fase inicial de planificación y diseño (aprox. 2 semanas en septiembre), la fase de pre-desarrollo técnico sin metodología (aprox. 3 semanas en octubre) y los 9 Sprints de desarrollo formal (aprox. 18 semanas), sumando un total de 23 semanas de trabajo efectivo. Esto supone un total de 460 horas de desarrollo.

Tomando como referencia el mercado laboral actual en la provincia de Burgos, el coste de mercado para un desarrollador de *software* Junior se sitúa alrededor de los 14€/hora (cálculo estimado partiendo de un salario base de unos 24.500€ brutos anuales dividido entre 1.750 horas laborables estándar por convenio).

$$460 \text{ horas} \times 14 \text{ €/hora} = 6,440 \text{ €}$$

Para calcular el coste de facturación real que tendría este desarrollo para una empresa, se debe añadir el Impuesto sobre el Valor Añadido (IVA) general aplicable en España (21 %):

$$6,440 \text{ €} + (6,440 \text{ €} \times 0,21) = 7,792,40 \text{ €}$$

Costes de Hardware y Software

Todo el desarrollo, prueba de contenedores y simulación algorítmica se ha ejecutado íntegramente de manera local empleando el equipo informático personal del autor. Al tratarse de un ordenador portátil amortizado, se establece un coste de *hardware* para la fase de desarrollo de 0€.

A nivel de herramientas lógicas, el proyecto hace uso exclusivo de librerías, *frameworks* (React, Node, PostgreSQL) e IDEs (Visual Studio Code) amparados por licencias de código abierto o gratuitas, resultando en un coste de licencias de *software* de 0€.

Costes de Infraestructura y Mantenimiento

Aunque la fase de desarrollo se ha ejecutado y simulado en local de manera gratuita, el despliegue del sistema AgroSkopos en un entorno B2B de producción comercial incurriría en costes fijos de infraestructura. Alojar la API REST y la base de datos PostgreSQL en un Servidor Privado Virtual (VPS) de gama de entrada en la nube, junto con el alojamiento estático del *frontend*, supondría un coste operativo estimado de 15€/mes, lo que equivale a un gasto de mantenimiento de 180€ anuales.

De este modo, el coste total presupuestado para la creación del prototipo inicial asciende a **7.792,40 €** (IVA incluido) imputables de forma directa al esfuerzo de ingeniería.

Viabilidad legal

Al tratarse de una aplicación de monitorización con persistencia de información y cuentas de usuario, el cumplimiento normativo es esencial.

Protección de Datos (RGPD)

El sistema requiere registro previo y almacena credenciales cifradas y datos básicos de los operarios para la asignación de misiones. El tratamiento de esta información cumple con el Reglamento General de Protección de Datos (RGPD), garantizando que las contraseñas no se guarden nunca en texto plano y limitando la recolección de información a la estrictamente necesaria para la operativa del negocio B2B.

Licenciamiento de Terceros

El desarrollo de AgroSkopos se ha apoyado en el uso de diversas bibliotecas, *frameworks* y herramientas de terceros. Para garantizar la viabilidad legal del proyecto, es fundamental analizar las licencias de estos componentes y asegurar que permiten su uso comercial y la creación de obras derivadas.

En la tabla ?? se presenta un resumen de las licencias de las principales tecnologías integradas en la arquitectura del sistema:

Herramienta / Librería	Uso en el proyecto	Licencia
React	Librería base de la interfaz de usuario (<i>frontend</i>)	MIT
Node.js & Express	Entorno de ejecución y <i>framework</i> del <i>backend</i>	MIT
PostgreSQL	Sistema de gestión de base de datos relacional	PostgreSQL
Leaflet	Librería cartográfica interactiva y mapas	BSD-2-Clause
Docker	Contenerización y despliegue virtualizado	Apache 2.0
Zustand	Gestión del estado global en el cliente	MIT
Prisma	ORM para la interacción tipada con la base de datos	Apache 2.0
Zod	Validación estricta de esquemas y tipos de datos	MIT
Vite	Herramienta de construcción y empaquetado del <i>frontend</i>	MIT
i18next	Internacionalización y soporte multilingüe	MIT
Recharts	Visualización de datos y generación de gráficas	MIT
Jest & Supertest	<i>Frameworks</i> para pruebas unitarias y de integración	MIT
Swagger (OpenAPI)	Documentación interactiva de la API REST	Apache 2.0

Tabla A.1: Resumen de licencias de las herramientas y tecnologías utilizadas

Como se observa detalladamente en la tabla, la gran mayoría de las tecnologías base operan bajo licencias altamente permisivas (MIT, Apache 2.0, BSD y PostgreSQL). Todas estas licencias son perfectamente compatibles entre sí y carecen de un carácter vírico estricto (a diferencia de licencias como GPL), lo que significa que no obligan a publicar como código abierto el *software* propietario que interactúa con ellas. Esta compatibilidad estructural avala la viabilidad legal plena de AgroSkopos, garantizando que la plataforma puede ser distribuida o explotada comercialmente en el sector B2B sin riesgo de infracción de propiedad intelectual heredada.

Selección de Licencia para la Documentación

Para la memoria principal y la documentación técnica del proyecto (incluyendo los anexos y diagramas), se ha seleccionado la licencia **Creative Commons Attribution 4.0 International (CC BY 4.0)**. Esta licencia ampara el trabajo literario fomentando la divulgación del conocimiento universitario, permitiendo a otros investigadores o alumnos compartir y adaptar el material para cualquier propósito, con la única condición indispensable de otorgar el crédito apropiado al autor original del documento.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este apéndice detalla la especificación de requisitos del sistema AgroSkopos. Se enumeran los objetivos generales que motivan el desarrollo, seguidos de un catálogo exhaustivo de los requisitos funcionales organizados por módulos. Finalmente, se detalla la especificación de los casos de uso principales, describiendo el flujo de interacción entre los distintos actores (Usuario, Operador, Administrador) y el sistema.

B.2. Objetivos generales

El principal objetivo que se planteó para este proyecto era desarrollar una solución que permita a una empresa o explotación agrícola manejar y gestionar un robot agrícola de forma centralizada y remota. Para lograr este fin, se han definido los siguientes objetivos secundarios:

- Desarrollar una plataforma centralizada y segura para la gestión de usuarios y roles dentro del entorno empresarial.
- Proveer capacidades de monitorización y telemetría en tiempo real, ofreciendo datos críticos del estado del robot y del terreno.
- Implementar un sistema de control bidireccional que permita la operación manual remota y la navegación autónoma.
- Facilitar la planificación y ejecución de misiones agrícolas autónomas mediante herramientas de geofencing.

B.3. Catálogo de requisitos

Módulo de Autenticación y Seguridad

- **RF-1.1 (Inicio de Sesión):** El sistema debe permitir a los empleados iniciar sesión introduciendo su nombre de usuario y contraseña.
- **RF-1.2 (Cierre de Sesión):** El sistema debe permitir a los usuarios cerrar su sesión de forma segura, invalidando el acceso.
- **RF-1.3 (Restricción de Registro):** El sistema no debe permitir el auto-registro público de usuarios. La creación de cuentas es exclusiva de la administración.
- **RF-1.4 (Gestión de Perfil Propio):** El sistema debe permitir a cualquier empleado autenticado modificar su contraseña.

Módulo de Administración de Cuentas

- **RF-2.1 (Creación de Empleados):** El sistema debe permitir al Administrador dar de alta nuevas cuentas de empleados en la plataforma.
- **RF-2.2 (Asignación de Roles):** El sistema debe permitir al Administrador asignar y modificar el nivel de privilegio de un usuario (Admin, Operador, Usuario).
- **RF-2.3 (Baja de Empleados):** El sistema debe permitir al Administrador deshabilitar o eliminar cuentas de empleados.
- **RF-2.4 (Listado de Usuarios):** El sistema debe proporcionar al Administrador una vista con todos los usuarios registrados y su rol actual.

Módulo de Monitorización y Telemetría

- **RF-3.1 (Posicionamiento GPS):** El sistema debe mostrar en un mapa interactivo la ubicación exacta del robot actualizada en tiempo real.
- **RF-3.2 (Estado de Energía):** El sistema debe mostrar el porcentaje de batería, si está cargando o descargando, y mostrar información sobre la potencia instantánea de generación solar.

- **RF-3.3 (Estado General):** El sistema debe informar sobre la actividad actual del robot (Ej: En espera, Trabajando, Regresando a base).
- **RF-3.4 (Lecturas Agronómicas):** El sistema debe mostrar al instante los valores simulados recogidos por los sensores del suelo (Humedad, Temperatura, pH, N-P-K y Radiación).

Módulo de Operación y Control Directo

- **RF-4.1 (Cambio de Modo):** El sistema debe permitir al Operador alternar entre los modos de funcionamiento del robot (Manual, Autónomo, Navegación).
- **RF-4.2 (Control Manual Remoto):** El sistema debe proporcionar un sistema de control mediante los botones de dirección de la interfaz o las flechas del teclado para conducir el robot en modo Manual.
- **RF-4.3 (Control de Velocidad):** El sistema debe permitir al Operador modificar la velocidad de desplazamiento del robot en tiempo real.
- **RF-4.4 (Navegación por Waypoints):** El sistema debe permitir encolar puntos en el mapa para que el robot navegue hacia ellos.
- **RF-4.5 (Pausa Rápida):** El sistema debe permitir pausar temporalmente cualquier movimiento del robot y poder reanudarlo.
- **RF-4.6 (Parada de Emergencia):** El sistema debe disponer de un control de emergencia que aborte inmediatamente la misión y detenga los motores.
- **RF-4.7 (Transmisión de Vídeo):** El sistema debe integrar un flujo de vídeo en tiempo real para facilitar la supervisión visual durante la operación y control del robot.

Módulo de Gestión de Misiones Autónomas

- **RF-5.1 (Gestión CRUD de Misiones):** El sistema debe permitir al Operador crear nuevas misiones, modificarlas, eliminarlas y consultar su histórico.

- **RF-5.2 (Dibujo de Área de Trabajo):** El sistema debe proveer herramientas para dibujar un polígono que delimite la parcela de trabajo.
- **RF-5.3 (Zonas de Exclusión):** El sistema debe permitir dibujar huecos o zonas de exclusión dentro del área principal donde el robot no deberá entrar.
- **RF-5.4 (Configuración de Parámetros de Ruta):** El sistema debe permitir ajustar el ancho entre pasada y pasada para la generación de la ruta en zig-zag.
- **RF-5.5 (Selección de Sensores):** El sistema debe permitir elegir qué datos se van a recoger durante la misión (ej. solo humedad, o reconocimiento rápido sin recogida de datos).
- **RF-5.6 (Umbral de Batería Inicial):** El sistema debe permitir establecer una batería mínima requerida para autorizar el inicio de la misión.
- **RF-5.7 (Lanzamiento de Misiones):** El sistema debe permitir iniciar la ejecución autónoma de una misión previamente configurada.

Módulo de Lógica Autónoma de Seguridad

- **RF-6.1 (Retorno Inteligente a Base):** El robot debe calcular continuamente la distancia hasta la estación de carga y retornar de forma autónoma cuando determine que la batería es estrictamente la necesaria para el trayecto de vuelta.
- **RF-6.2 (Reanudación Post-Carga):** El sistema debe volver al punto de interrupción para continuar la misión una vez recargado al 100 %.
- **RF-6.3 (Reposo Seguro):** El sistema debe enviar automáticamente al robot a la base de carga si permanece inactivo durante un periodo prolongado.

Módulo de Análisis de Datos Agronómicos

- **RF-7.1 (Exportación de Datos):** El sistema debe permitir la exportación de los datos recogidos y análisis mostrados a formatos estándar (PDF y CSV).

- **RF-7.2 (Filtros Temporales y por Misión):** El sistema debe incluir un calendario para filtrar los datos mostrados indicando fecha, hora y misión específica.
- **RF-7.3 (Visualización de Gráficas):** El sistema debe generar gráficas temporales con un único parámetro o gráficas comparativas cruzando dos variables agronómicas.
- **RF-7.4 (Visualización Tabular):** El sistema debe presentar una tabla detallada con todos los datos crudos recogidos, incluyendo la hora exacta y la localización de la muestra.
- **RF-7.5 (Geolocalización de Muestras):** El sistema debe permitir al usuario conocer la localización en el mapa de cada dato individual recogido.
- **RF-7.6 (Mapa de Puntos y Medias por Zona):** El sistema debe mostrar un mapa con los puntos donde se tomaron medidas, permitiendo agrupar/filtrar por zona para visualizar la media aritmética de los valores recogidos en dicha área.

Módulo de Usabilidad e Integración

- **RF-8.1 (Internacionalización):** El sistema debe permitir al usuario cambiar el idioma de la interfaz gráfica en tiempo real, soportando al menos los idiomas español, inglés y portugués.
- **RF-8.2 (Aplicación Web Progresiva):** El sistema debe ofrecer la capacidad de ser instalado como una aplicación nativa en dispositivos móviles (PWA) para facilitar su operabilidad a pie de campo.
- **RF-8.3 (Documentación Interactiva de API):** El servidor debe exponer una interfaz web donde los desarrolladores y administradores puedan explorar y probar interactivamente los *endpoints* del sistema (Swagger).

Requisitos No Funcionales

- **RNF-1 (Rendimiento y Latencia):** El sistema de telemetría y control remoto debe operar con una latencia mínima, garantizando una respuesta fluida a los comandos manuales y a la recepción de datos en tiempo real.

- **RNF-2 (Escalabilidad de Sistema):** La arquitectura del servidor y la base de datos deben permitir la escalabilidad para gestionar un alto volumen de datos históricos y soportar el control de múltiples robots de forma simultánea en el futuro.
- **RNF-3 (Portabilidad y Despliegue):** El sistema debe estar estructurado en contenedores virtuales que aislen los diferentes servicios, garantizando un despliegue rápido y reproducible en cualquier entorno o servidor.
- **RNF-4 (Protección de Red y API):** Las interfaces de comunicación deben incorporar mecanismos robustos de seguridad (como limitación de tasa de peticiones, control de origen cruzado y fortificación de cabeceras) para prevenir abusos y ataques externos.
- **RNF-5 (Integridad de Transacciones):** Las operaciones críticas que requieran múltiples escrituras deben estar protegidas mediante transacciones de base de datos para asegurar que no quede información corrupta o huérfana en caso de error.
- **RNF-6 (Usabilidad Responsiva):** La interfaz de usuario debe ser completamente adaptativa y ofrecer notificaciones claras, posibilitando su uso seguro e intuitivo tanto en monitores de escritorio como en dispositivos táctiles en el campo.
- **RNF-7 (Calidad de Código y Análisis Estático):** El código fuente debe someterse a análisis estático continuo para detectar vulnerabilidades, *code smells* y garantizar el cumplimiento de métricas de calidad predefinidas.
- **RNF-8 (Integración Continua):** La adición de nuevas características debe estar respaldada por tuberías de integración continua (CI) que automaticen la validación del código antes de ser desplegado.
- **RNF-9 (Fiabilidad y Cobertura de Pruebas):** Los módulos deben contar con una batería de pruebas unitarias y de integración automatizadas para prevenir regresiones y asegurar su correcto funcionamiento.
- **RNF-10 (Validación Estricta de Datos):** El sistema debe implementar mecanismos rigurosos de validación de esquemas en todas las entradas de datos para prevenir errores de tipo y fallos en tiempo de ejecución.

- **RNF-11 (Documentación de Código Autogenerada):** El código fuente debe estar debidamente documentado mediante un formato estandarizado (como JSDoc) que permita la generación automática y actualización continua de la documentación técnica.

B.4. Especificación de requisitos

A continuación se detallan los casos de uso principales identificados para el sistema AgroSkopos.

Antes de detallar las interacciones de cada caso de uso, es necesario definir los **Actores del Sistema**. El sistema implementa un modelo de permisos jerárquico e incremental basado en los siguientes roles:

- **Usuario:** Dispone de acceso de solo lectura. Puede acceder a la plataforma web para monitorizar la telemetría en tiempo real, visualizar la posición del robot en el mapa y consultar el histórico de datos y gráficas agronómicas. No tiene permisos para alterar el estado del sistema ni enviar comandos al robot.
- **Operador:** Hereda todos los permisos de visualización del perfil *Usuario*. Además, está capacitado para tomar el control activo del vehículo. Puede conducir el robot en modo manual, dibujar áreas de trabajo, configurar parámetros y lanzar misiones autónomas.
- **Administrador (Admin):** Posee el control total del sistema. Hereda las capacidades operativas del *Operador* y añade privilegios exclusivos de gestión interna, como la creación, modificación y eliminación de cuentas de empleados, así como la asignación de roles.

CU-1	Iniciar sesión
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Usuario, Operador, Administrador
Requisitos asociados	RF-1.1
Descripción	Permite a un empleado acceder a la plataforma mediante sus credenciales.
Precondición	El usuario debe estar registrado por un Administrador y disponer de credenciales válidas.
Acciones	1. El usuario accede a la página de inicio de sesión. 2. El sistema solicita correo electrónico y contraseña. 3. El usuario introduce sus datos y pulsa el botón de acceso. 4. El sistema valida las credenciales y devuelve un token de acceso (JWT). 5. El sistema redirige al usuario a la vista principal (Dashboard).
Postcondición	El usuario queda autenticado y tiene acceso a las funciones permitidas por su rol.
Excepciones	Credenciales incorrectas: El sistema deniega el acceso y muestra un mensaje de error.
Importancia	Alta

Tabla B.1: CU-1 Iniciar sesión.

CU-2	Gestionar cuentas de empleado
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Administrador
Requisitos asociados	RF-2.1, RF-2.2, RF-2.3, RF-2.4
Descripción	Permite al Administrador dar de alta, modificar roles o eliminar empleados del sistema.
Precondición	El usuario debe estar autenticado y tener el rol <i>Admin</i> .
Acciones	1. El Administrador navega a la sección de "Gestión de Usuarios". 2. El sistema muestra un listado con los usuarios actuales. 3. El Administrador selecciona crear un nuevo usuario o editar uno existente. 4. El Administrador introduce/modifica los datos (nombre, correo, rol) y guarda. 5. El sistema actualiza la base de datos y refleja los cambios en el listado.
Postcondición	Las credenciales y privilegios del usuario afectado se actualizan en el sistema.
Excepciones	Correo duplicado al crear usuario: El sistema informa de que la cuenta ya existe.
Importancia	Alta

Tabla B.2: CU-2 Gestionar cuentas de empleado.

CU-3	Control manual del robot
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Operador, Administrador
Requisitos asociados	RF-4.1, RF-4.2, RF-4.3
Descripción	El Operador toma el control directo del movimiento usando botones en pantalla o teclado, pudiendo regular la velocidad.
Precondición	El usuario debe tener rol <i>Operador</i> o <i>Admin</i> . El robot debe tener conexión y batería suficiente.
Acciones	1. El Operador establece el modo de funcionamiento del robot a "Manual". 2. El Operador ajusta el nivel de velocidad deseado. 3. El Operador presiona las flechas de dirección en la interfaz o su teclado. 4. El sistema envía vectores de velocidad y dirección mediante WebSockets al simulador/robot. 5. El robot procesa el movimiento y actualiza su posición en el mapa interactivo.
Postcondición	El robot se desplaza según las indicaciones del Operador.
Excepciones	Pérdida de conexión: El robot se detiene automáticamente por seguridad.
Importancia	Alta

Tabla B.3: CU-3 Control manual del robot.

CU-4	Diseñar y ejecutar misión autónoma
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Operador, Administrador
Requisitos asociados	RF-5.2, RF-5.3, RF-5.4, RF-5.5
Descripción	El Operador planifica al detalle una ruta de barrido definiendo parcelas, huecos y sensores a activar.
Precondición	Rol <i>Operador</i> o <i>Admin.</i> El robot está en espera y supera el umbral mínimo de batería.
Acciones	1. El Operador accede a la herramienta de Misiones y dibuja el polígono externo de la parcela en el mapa. 2. El Operador delimita sub-polígonos como "zonas de exclusión" dentro de la parcela. 3. El Operador configura el espaciado de las pasadas en zig-zag y los sensores a utilizar. 4. El sistema calcula una ruta óptima evitando los huecos marcados. 5. El Operador hace clic en "Iniciar Misión". El robot cambia a modo <i>Autónomo</i> y sigue la trayectoria.
Postcondición	La misión se registra en el historial como <i>En curso</i> y el robot opera recolectando los datos seleccionados.
Excepciones	Batería inferior al mínimo de seguridad configurado: El sistema rechaza iniciar la misión.
Importancia	Muy Alta

Tabla B.4: CU-4 Diseñar y ejecutar misión autónoma.

CU-5	Retorno automático por cálculo de distancia
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Sistema (Automático)
Requisitos asociados	RF-6.1, RF-6.2
Descripción	El sistema aborta la misión justo a tiempo para volver a la base evaluando la distancia restante y batería.
Precondición	El robot está ejecutando una misión autónoma lejos de la estación de carga.
Acciones	1. El sistema calcula continuamente la distancia euclídea y el coste energético hasta la base. 2. El sistema detecta que la batería restante es la mínima indispensable para el viaje de vuelta. 3. El sistema pausa la misión guardando en memoria el estado y la coordenada de interrupción. 4. El sistema activa el modo Retorno a Basez el robot navega a la estación. 5. Al alcanzar el 100 % de carga, el robot navega de vuelta a la coordenada y reanuda la misión.
Postcondición	El robot gestiona su energía para no quedarse sin batería en campo abierto.
Excepciones	Obstáculo insalvable en el retorno: El robot detiene motores y pide intervención humana.
Importancia	Alta

Tabla B.5: CU-5 Retorno automático por cálculo de distancia.

CU-6	Análisis integral de datos agronómicos
Versión	1.0
Autor	Andrés Vera Cachoo
Actores	Usuario, Operador, Administrador
Requisitos asociados	RF-7.2, RF-7.3, RF-7.4, RF-7.6
Descripción	El usuario filtra, visualiza y exporta métricas recogidas en campo bajo distintas perspectivas.
Precondición	Existen misiones completadas con recolección de datos en la base de datos.
Acciones	1. El usuario selecciona fechas en el calendario y filtra por misiones específicas. 2. El sistema muestra un listado de muestras recogidas (tabla) con hora y posición. 3. El usuario genera una gráfica comparativa superponiendo los datos de dos parámetros. 4. El usuario cambia a la vista de "Mapa de Puntos" delimita una zona concreta. 5. El sistema calcula y muestra la media aritmética de esa zona para los sensores marcados. 6. El usuario hace clic en ".Exportar a PDF." obteniendo un informe del análisis.
Postcondición	El usuario obtiene visualizaciones útiles de los datos recopilados por el robot.
Excepciones	Filtros sin resultados: El sistema avisa de que no hay datos en ese rango de tiempo/zona.
Importancia	Alta

Tabla B.6: CU-6 Análisis integral de datos agronómicos.

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo describe en detalle las decisiones técnicas y la arquitectura adoptada durante la construcción de AgroSkopos. El objetivo de esta fase de diseño es transformar los requisitos analizados en especificaciones tangibles de bases de datos, estructuración algorítmica y despliegue de *software*. Se aborda, de forma escalonada, la estructuración de la persistencia de datos (esquema relacional), el diseño procedimental de la lógica algorítmica, y por último, la arquitectura global del sistema mediante diagramas.

C.2. Diseño de datos

Arquitectura y justificación de la base de datos

El diseño arquitectónico de la persistencia de datos persigue como premisa fundamental la **simplicidad y el pragmatismo**. En lugar de sobredimensionar el esquema con decenas de tablas altamente normalizadas que acabarían fragmentando la telemetría, se ha optado intencionadamente por un modelo de datos conciso y robusto compuesto por seis entidades clave. Esta simplificación estratégica está firmemente justificada por dos motivos fundamentales: en primer lugar, maximiza el rendimiento y la velocidad de escritura, un factor crítico cuando el robot debe enviar ráfagas de telemetría de sensores por segundo; y en segundo lugar, reduce la complejidad computacional en el *backend* a la hora de realizar cruces de información (JOINS), facilitando su mantenimiento y lectura.

La capa de persistencia se asienta sobre un sistema gestor de bases de datos relacionales PostgreSQL. La interacción entre el *backend* (Node.js) y la base de datos no se realiza mediante consultas SQL crudas, sino que se gestiona íntegramente a través de un ORM (*Object-Relational Mapping*). La adopción de un ORM abstrae la complejidad de la base de datos, previniendo vulnerabilidades críticas como la inyección SQL, y permite manipular la información utilizando paradigmas orientados a objetos directamente en el código de la aplicación.

Para este proyecto, la elección tecnológica ha recaído sobre **Prisma ORM**. Frente a alternativas más antiguas como Sequelize o TypeORM, Prisma destaca por autogenerar un cliente de base de datos fuertemente tipado basándose en un único archivo de configuración declarativo (`schema.prisma`). Esto garantiza que cualquier discrepancia entre el modelo de datos y el código se detecte de forma inmediata en tiempo de compilación (evitando fallos inesperados en producción) y centraliza la gestión de migraciones, simplificando radicalmente la evolución del proyecto y mejorando la experiencia de desarrollo.

Entidades del modelo

El modelo de datos se ha diseñado buscando el equilibrio comentado entre normalización estructural y rendimiento masivo. A continuación, se detallan las entidades principales que componen el sistema:

- **usuarios:** Almacena las credenciales cifradas y los perfiles de acceso (roles, avatares) del personal de la explotación agraria. Mantiene una relación *1 a N* con *misiones* (quién planificó la tarea) y con *ejecuciones_mision* (quién dio la orden de ejecución).
- **misiones:** Define los parámetros estáticos, espaciales y logísticos de una tarea agronómica. Incluye campos geométricos almacenados en formato JSON (*area_trabajo*, *puntos_interes*) así como requisitos operativos (ancho de trabajo, batería mínima requerida). Registra el *usuario_id* del creador.
- **ejecuciones_mision:** Actúa como entidad transaccional que registra las métricas temporales de una misión al ser desplegada en el terreno. Controla el estado actual de la ejecución, el progreso, la fecha de inicio/fin y la distancia real recorrida. Mantiene una relación *1 a N* con la tabla *misiones* y asocia opcionalmente el *usuario_id* del operario que inició la tarea.

- **robot_datos:** Diseñada para soportar una alta carga de inserciones, esta tabla almacena la telemetría agronómica capturada por los sensores en tiempo real durante una ejecución (niveles de nitrógeno, fósforo, potasio, pH, temperatura del suelo y radiación solar), georreferenciando cada lectura con coordenadas espaciales. Mantiene una relación *1 a N* con *ejecuciones_mision*.
- **robot_estado:** Entidad orientada a reflejar el *hardware* en tiempo real. En ella, el robot reporta periódicamente su telemetría de bajo nivel (coordenadas actuales, velocidad, *heading*, temperatura y voltaje de batería).
- **historial_energia:** Persiste métricas históricas de consumo eléctrico y captación energética del panel solar para permitir el análisis a largo plazo del rendimiento y la autonomía del prototipo.

A continuación, en la figura C.1, se expone el Diagrama Entidad-Relación (ER) lógico que ilustra la cardinalidad e interacción semántica entre los dominios principales. Cabe destacar que la estructura visual de este diagrama ha sido autogenerada de manera a partir del esquema de configuración de Prisma ORM.

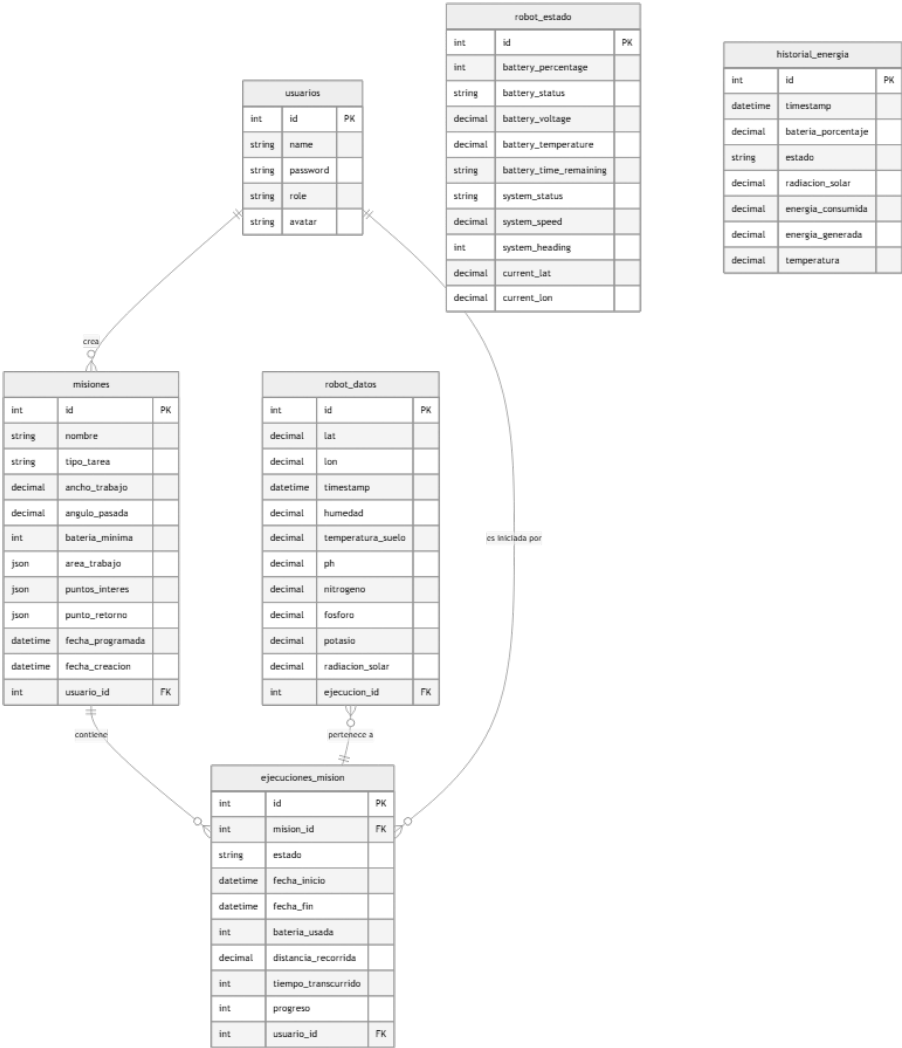


Figura C.1: Diagrama Entidad-Relación Lógico del Sistema

C.3. Diseño procedimental

Este apartado desciende desde la abstracción de los modelos relacionales hacia la sala de máquinas del *software*. Aquí se detallan las especificaciones técnicas, los patrones de diseño arquitectónicos adoptados para resolver problemas recurrentes y la lógica algorítmica fundamental (el motor de simulación) que gobierna el comportamiento interno de AgroSkopos.

Patrones de diseño utilizados

El código de la aplicación se ha estructurado siguiendo paradigmas consolidados para garantizar mantenibilidad, alta cohesión y bajo acoplamiento tanto en el *backend* como en el *frontend*.

Patrones en el servidor (Node.js)

- **Arquitectura Multicapa (Controller-Service-Repository):** Las peticiones HTTP entrantes son interceptadas por un *Router* que delega en un *Controller*. Este último se apoya en un *Service* que contiene la lógica de negocio pura, aislando las interacciones de base de datos a la capa del repositorio (Prisma). Esta separación previene la existencia de controladores engordados (*fat controllers*).
- **Patrón Singleton y Motor de Estado:** El corazón del sistema (el archivo `simulator.js`) opera bajo un patrón Singleton. Al importarse como módulo de Node.js, mantiene una única instancia en memoria (un cierre léxico o *closure*) que preserva el estado global de la simulación del robot para todos los usuarios concurrentes, sin necesidad de consultar el disco continuamente.
- **Patrón Observer (Pub-Sub):** Implementado a través de WebSockets (*Socket.io*). El servidor actúa como publicador emitiendo métricas (*robot:new_data*), mientras que los clientes web actúan como suscriptores reactivos, actualizando sus interfaces sin peticiones activas (*polling*).
- **Middlewares y validación de entrada:** Se emplea un patrón de interceptación en las rutas de Express mediante validadores esquemáticos (Zod). Esto asegura que ninguna petición con cuerpo malformado o sin *token* JWT alcance la lógica de dominio.

Patrones en el cliente web (React)

- **Patrón Store / Arquitectura Flux:** La aplicación abandona el antipatrón de *prop-drilling* en favor de **Zustand**. Se definen *stores* globales (ej. `robotStore.js`) que inyectan el estado reactivo de la telemetría directamente en los componentes que lo necesitan (el mapa o las gráficas), aislando la mutación de estado de la capa de renderizado.
- **Context API / Provider:** Para dependencias estáticas que envuelven toda la aplicación, como la sesión de usuario, se emplea **AuthContext**,

permitiendo proteger rutas privadas a nivel de enrutador (*React Router*).

Algoritmia de simulación pesada

A diferencia de los sistemas de gestión tradicionales orientados a modelos anémicos, el *backend* de AgroSkopos asume el reto de emular un entorno de agricultura de precisión interactivo en tiempo real mediante dos lógicas circulares:

1. **Bucle de Físicas (Energía y Batería):** El servidor realiza un cálculo continuo de recarga solar. La función `calculateSolarRadiation(date)` modela matemáticamente la posición del sol basándose en la fecha, hora y coordenadas geográficas para devolver un valor en W/m^2 , que a su vez se traduce en amperaje de recarga. Este proceso combate en paralelo con el consumo paramétrico derivado del estado del motor (IDLE vs WORKING).
2. **Bucle Agronómico y Búfer Circular:** Durante el *pathfinding* (`generateCoveragePath`), el robot emite puntos GPS interpolados. En cada coordenada, un algoritmo pseudoaleatorio criptográficamente seguro inyecta valores de NPK y pH acordes al bioma simulado. Para prevenir la sobrecarga de la base de datos PostgreSQL, la simulación opera como un **búfer circular**, ejecutando de forma asíncrona sentencias SQL de limpieza (`DELETE ... NOT IN (SELECT ... LIMIT N)`) que preservan únicamente el histórico reciente, optimizando drásticamente el consumo de RAM.

Flujos principales del sistema

A continuación, se ilustran a través de diagramas de secuencia UML los cinco flujos operacionales más críticos de la arquitectura.

Flujo de Autenticación y Autorización

El primer punto de contacto de cualquier operario con el sistema es el inicio de sesión (figura C.2). Los pasos son los siguientes:

1. El usuario introduce sus credenciales en el formulario de la aplicación web y el *frontend* envía la petición POST al *backend*.

2. El *backend* (Express) busca al usuario en la base de datos a través del ORM Prisma.
3. Se verifica la validez de la contraseña comparándola con el *hash* almacenado utilizando la librería *bcrypt*.
4. Si las credenciales son válidas, el sistema genera y firma un *JSON Web Token* (JWT) que se devuelve al cliente.
5. El *Context API* de React almacena el *token* en memoria local, utilizándolo a partir de ese momento para autorizar de forma segura las futuras peticiones al API.

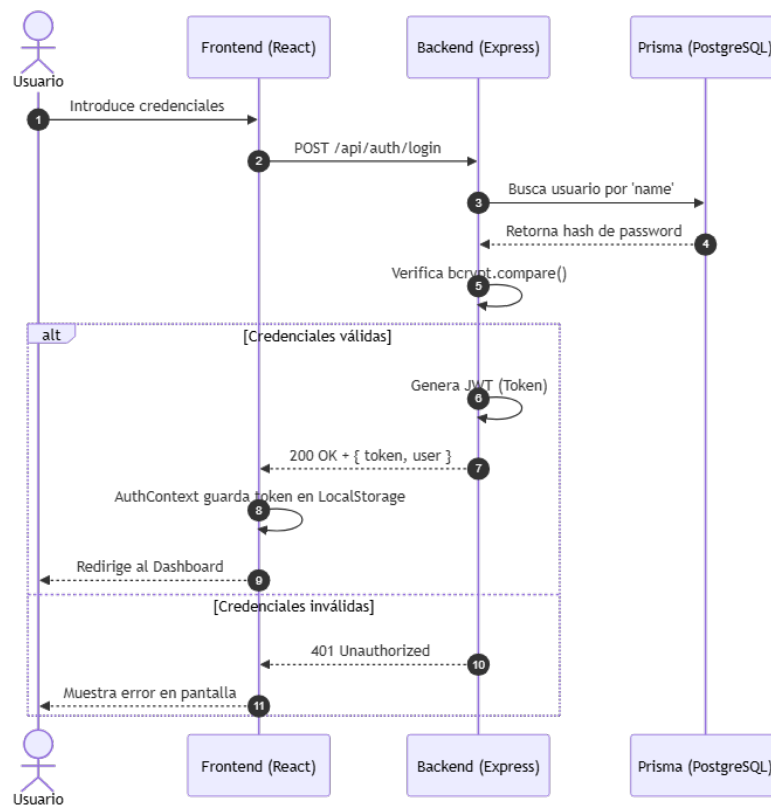


Figura C.2: Diagrama de Secuencia: Flujo de Autenticación

Flujo de Planificación de Misión Agrícola

Antes de ejecutar una tarea, el sistema requiere una planificación espacial (figura C.3). El proceso de creación es el siguiente:

1. El operario utiliza la capa interactiva del mapa en React para trazar visualmente un polígono cerrado sobre la parcela.
2. El *frontend* recolecta estos puntos geográficos, calcula el área teórica y los formatea al estándar *GeoJSON*.
3. El usuario completa el resto del formulario (nombre, tipo de cultivo, tarea) y lo envía al servidor.
4. El API intercepta el paquete y un *middleware* valida la estandarización geométrica de las coordenadas utilizando *Zod*.
5. Si la validación es exitosa, se persiste la nueva misión en la base de datos y se redirige al usuario a la lista de tareas pendientes.

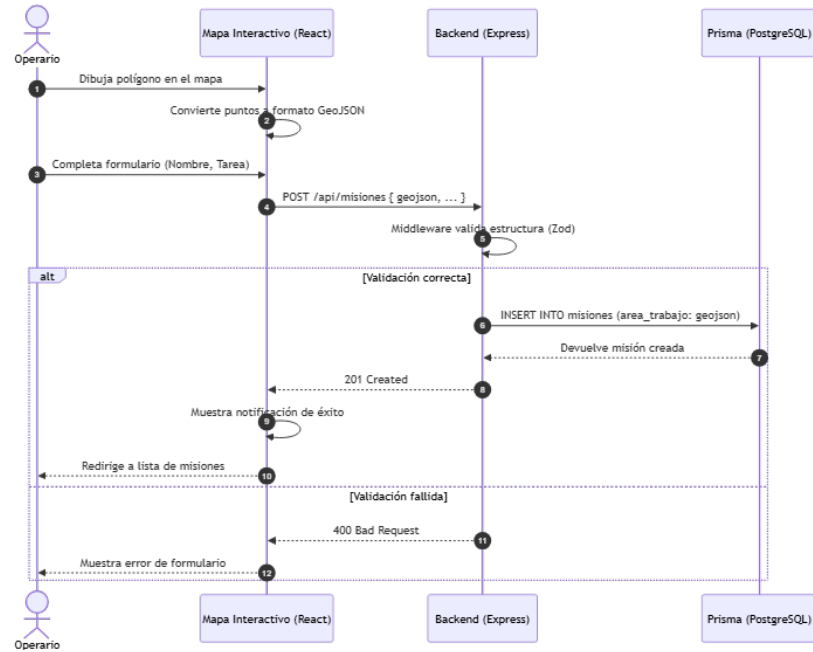


Figura C.3: Diagrama de Secuencia: Planificación de Cobertura Agrícola

Flujo de Ejecución de Misión

La transición de una misión desde su fase de planificación estática hasta su ejecución dinámica en el terreno se detalla en la figura C.4:

1. El operario localiza una misión en estado pendiente y pulsa el botón de *Iniciar* desde el panel de control.

2. El *backend* recibe la instrucción y comprueba las precondiciones operativas, asegurando que el robot cuente con la batería mínima requerida.
3. Se genera un nuevo registro en la tabla `ejecuciones_mision` para trazar el marco temporal de la tarea.
4. El servidor activa el contexto en la memoria del simulador y el motor matemático calcula la ruta de cobertura de área (*pathfinding*).
5. Se notifica instantáneamente a todos los clientes a través del bus de eventos (Socket.io). El gestor de estado (*Zustand*) actualiza la interfaz en vivo mostrando al robot desplazándose.

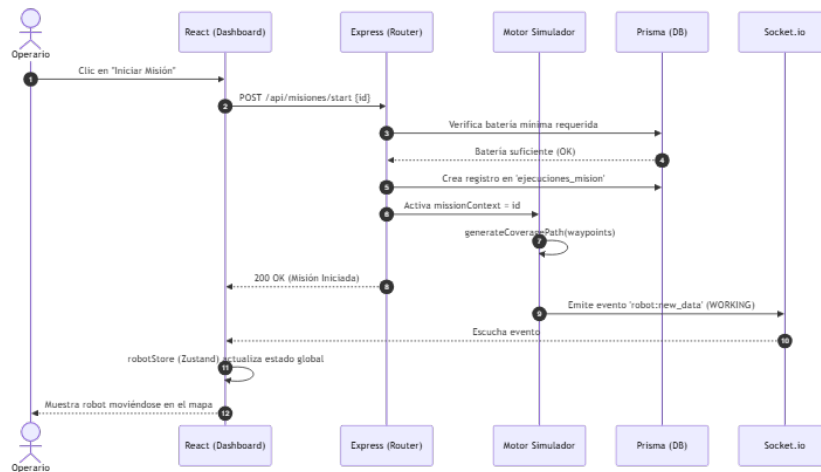


Figura C.4: Diagrama de Secuencia: Inicio de una Misión Agrícola

Ciclo de Vida de Telemetría (Real-Time)

El flujo más recurrente y pesado del proyecto es la captación continua de telemetría (figura C.5), que actúa como el "latido" del sistema:

1. El intervalo asíncrono interno de Node.js se dispara periódicamente, actuando como el reloj maestro de la simulación.
2. El motor de físicas recalcula la recarga de los paneles solares y el drenaje de la batería basándose en el clima y el estado actual.
3. El motor agronómico interpola la nueva posición GPS en el terreno y simula lecturas de NPK y pH inyectando valores semi-aleatorios atados a una semilla criptográfica.

4. Los datos resultantes se persisten de forma masiva en PostgreSQL, activando en paralelo el algoritmo de limpieza de búfer circular.
5. Finalmente, el servidor emite el paquete JSON comprimido por Web-Sockets. React recibe los paquetes y fuerza el redibujado instantáneo de la posición y mapas de calor.

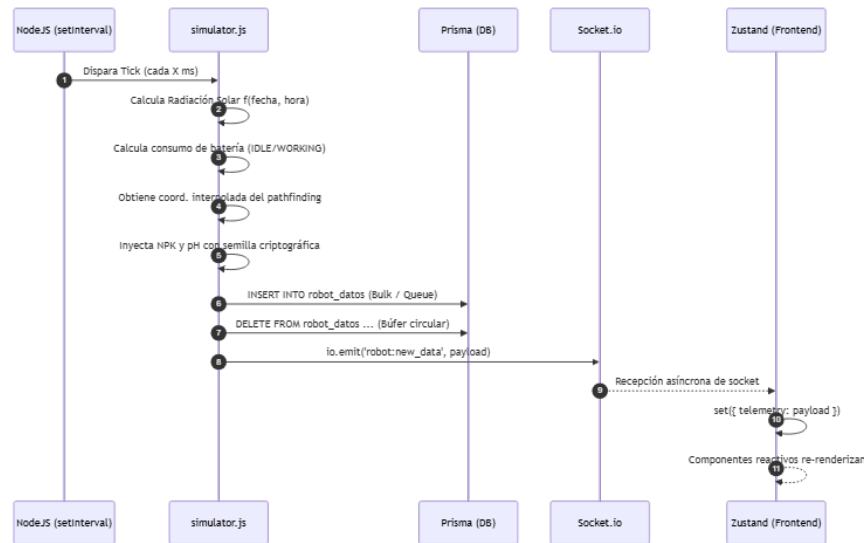


Figura C.5: Diagrama de Secuencia: Ciclo de Telemetría Asíncrona

Consulta de Histórico y Análisis Energético

Para permitir la toma de decisiones, la aplicación consolida el rendimiento pasado del robot en gráficas interactivas (figura C.6):

1. El administrador accede a la sección de analíticas desde la barra de navegación lateral.
2. React despacha peticiones HTTP REST concurrentes al *backend* solicitando datos agregados.
3. Prisma ejecuta sentencias complejas de extracción en la tabla de `robot_datos` y en `historial_energia`.
4. El servidor Node.js unifica, pagina y filtra las matrices de resultados, devolviéndolas serializadas.

5. El componente de *frontend* formatea la estructura de los datos para inyectarlos en la librería visual Recharts, renderizando finalmente las curvas de rendimiento.

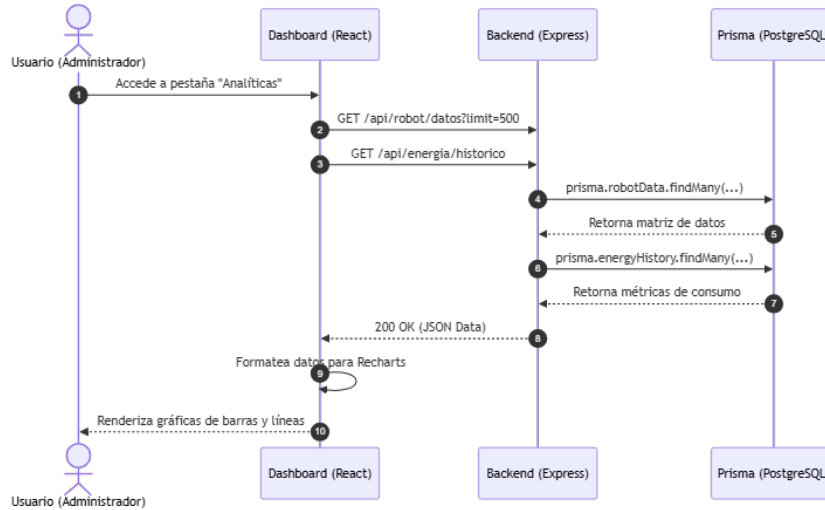


Figura C.6: Diagrama de Secuencia: Flujo de Visualización Analítica

C.4. Diseño arquitectónico

En esta sección se define la estructura global del sistema, la organización de sus repositorios lógicos y las reglas que rigen la comunicación entre ellos. Para AgroSkopos, se ha optado por una arquitectura de alto rendimiento orientada a eventos y procesamiento geoespacial, desmarcándose fuertemente de los sistemas de gestión tradicionales.

Arquitectura Global e Infraestructura

El sistema implementa un modelo de arquitectura cliente-servidor completamente desacoplado y regido por un **Protocolo Dual**. A diferencia de las aplicaciones web convencionales basadas estrictamente en transferencias de estado (REST), donde el cliente debe sondear continuamente al servidor en busca de cambios (*polling*), AgroSkopos utiliza un enfoque híbrido para optimizar la carga de red:

- **Canal transaccional (HTTP/REST):** Empleado para operaciones de ciclo de vida largo, estáticas y que requieren validación rigurosa,

como la autenticación de operarios, la consulta de históricos o la inserción de nuevas misiones en la base de datos.

- **Canal de flujo continuo (WebSockets):** Empleado para mantener una conexión TCP persistente, bidireccional y *stateful* entre el servidor y los clientes. Este patrón (Pub-Sub) permite que el motor de simulación emita un *stream* ininterrumpido de telemetría a alta frecuencia, garantizando que el mapa interactivo se mueva en tiempo real con una latencia mínima.

Para facilitar la comprensión visual de estas interacciones, se han elaborado dos esquemas. La figura C.7 detalla las tecnologías lógicas empleadas en cada capa de la aplicación. Por otro lado, la figura C.8 ilustra la arquitectura de despliegue físico.

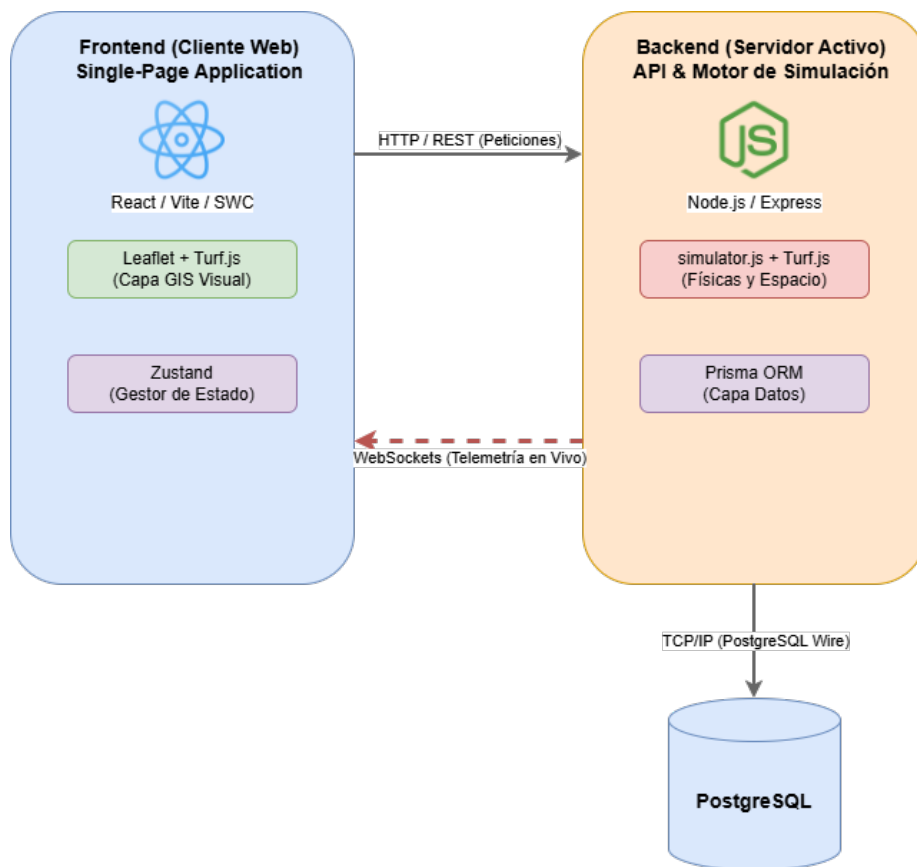


Figura C.7: Diagrama de Arquitectura de Software: Interacción entre Cliente, Servidor y Base de Datos

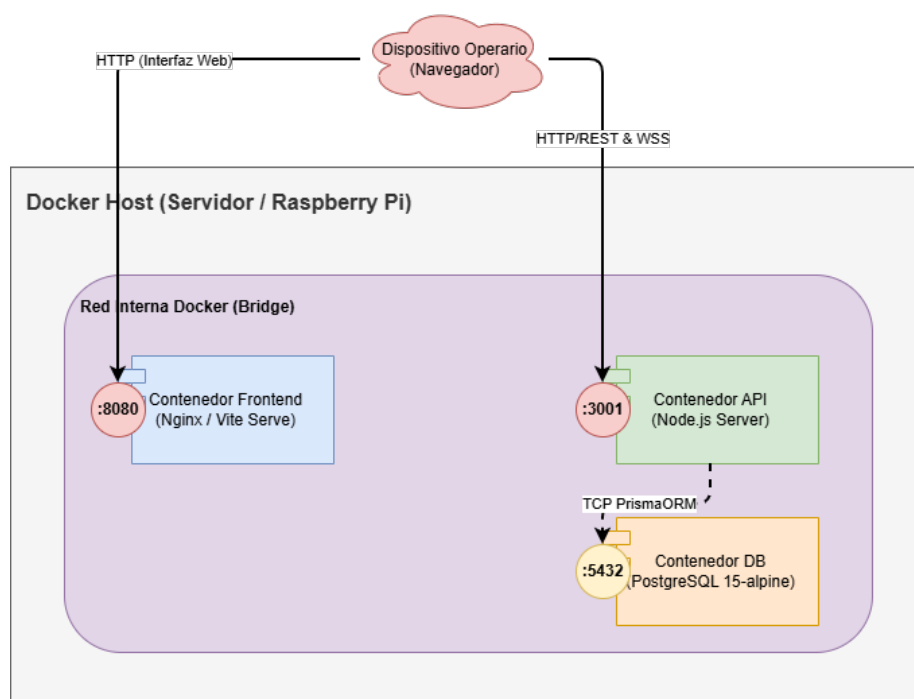


Figura C.8: Diagrama de Infraestructura y Despliegue: Contenedores Docker

A nivel de infraestructura, el sistema se apoya íntegramente en la contenerización mediante Docker. Al orquestar los servicios con `docker-compose`, se levantan tres contenedores lógicos aislados: la base de datos relacional (PostgreSQL), el motor API (Node.js) y el servidor de estáticos del *frontend*. Esta separación garantiza que los entornos de desarrollo, pruebas y producción sean idénticos, evitando el clásico problema de "funciona en mi máquina".

Arquitectura del Backend (Node.js)

El servidor central de AgroSkopos está desarrollado sobre el entorno de ejecución asíncrono Node.js utilizando el *framework* Express. La decisión de usar Node.js radica en su arquitectura no bloqueante (basada en el *Event Loop*), la cual es excepcionalmente brillante para manejar miles de conexiones simultáneas por WebSockets sin agotar los hilos del procesador.

El *backend* abandona el concepto pasivo habitual para convertirse en un **Motor Activo**. Normalmente, un servidor web permanece inactivo hasta que recibe una petición HTTP. Sin embargo, este servidor ejecuta un bucle de simulación interno (`simulator.js`) que corre de fondo ininterrumpidamente,

calculando variables agronómicas, consumo de batería y geolocalización, independientemente de si hay clientes conectados o no.

Las tecnologías clave que sostienen la arquitectura del servidor son:

- **Enrutamiento y Control (Express):** Gestiona el ciclo de vida de las peticiones HTTP y aplica las políticas de control de acceso cruzado (CORS).
- **Capa de Validación Rigurosa (Zod):** Se adopta una política de *fail-fast*. Antes de que cualquier paquete de datos alcance los controladores, los *middlewares* interceptan el *payload* y lo comparan contra esquemas matemáticos estrictos utilizando la librería Zod. Esto previene eficazmente inyecciones NoSQL y asegura que las coordenadas geográficas mantengan su integridad.
- **Análisis Espacial (Turf.js):** Dado que la aplicación es altamente geográfica, el servidor incorpora `@turf/turf` para realizar proyecciones geométricas y calcular áreas de polígonos interceptados sin necesidad de delegar este cálculo a PostGIS, reduciendo la carga transaccional de la base de datos.
- **Capa de Persistencia (Prisma ORM):** En contraposición a escribir sentencias SQL en crudo, se utiliza Prisma como Mapeador Objeto-Relacional. Prisma garantiza seguridad de tipos (Type-Safety) durante las operaciones CRUD, evitando errores humanos durante el desarrollo e inyecciones SQL en tiempo de ejecución.
- **Auto-Documentación Estándar (Swagger/OpenAPI):** La interfaz del API REST no es opaca. Se han integrado las librerías `swagger-jsdoc` y `swagger-ui-express` para generar en tiempo real una interfaz interactiva de documentación técnica. Esto permite auditar los *endpoints*, *payloads* y códigos de estado HTTP directamente desde el navegador, cumpliendo con los estándares de diseño *API-First*.

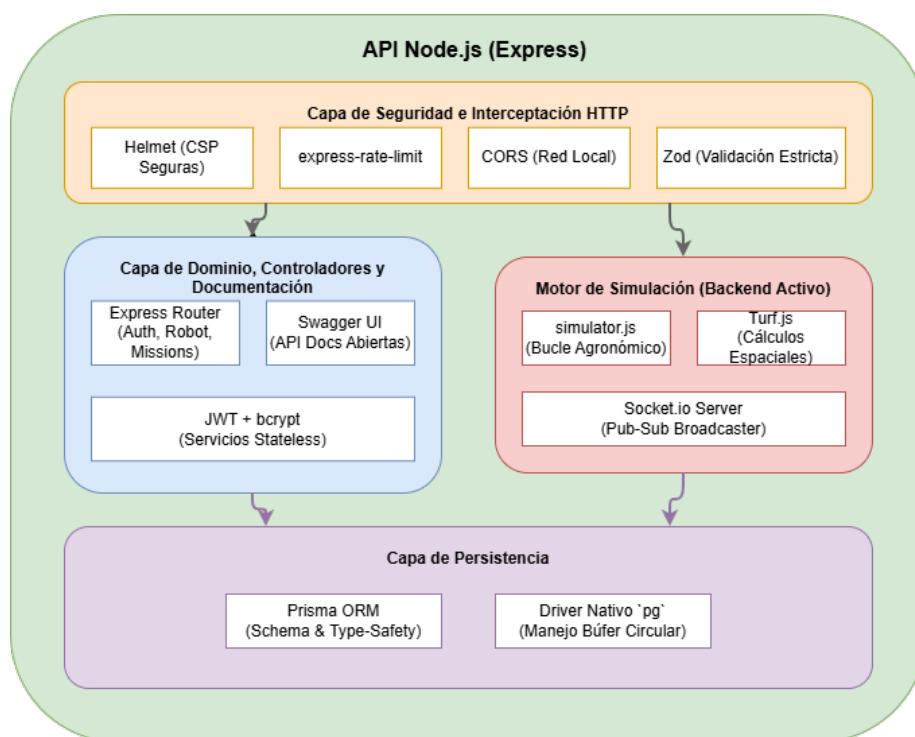


Figura C.9: Diagrama de Arquitectura Interna del Backend (Node.js y Motor Activo)

Arquitectura del Frontend (React)

El cliente web está diseñado bajo el paradigma de las *Single-Page Applications* (SPA). En lugar de renderizar el HTML en el servidor para cada clic, la interfaz se carga una sola vez en el navegador y muta de forma dinámica, ofreciendo una experiencia fluida equivalente a la de un software de escritorio nativo.

Para lograr tiempos de compilación instantáneos, se ha prescindido de los empaquetadores heredados como Webpack en favor de **Vite**. Acoplado con el compilador SWC (escrito en Rust), la aplicación se levanta en milisegundos durante el desarrollo.

La arquitectura del cliente se subdivide en las siguientes áreas tecnológicas de alta complejidad:

- **Subsistema de Información Geográfica (Capa GIS):** Constituye el núcleo central visual de la plataforma. El lienzo del mapa está motorizado por `leaflet` e integrado en el ciclo de vida de los

componentes mediante `react-leaflet`. La trazabilidad interactiva (cuando el operario dibuja el área de trabajo) recae sobre la librería `@geoman-io/leaflet-geoman-free`. Además, las capas topológicas que representan las lecturas agronómicas del simulador son interpoladas visualmente mediante `leaflet.heat`.

- **Gestor de Estado Global (Zustand):** Uno de los mayores retos arquitectónicos de una SPA compleja es evitar el *prop-drilling* (pasar datos entre docenas de componentes anidados). En lugar de emplear Redux, que añade una verbosidad excesiva y ralentiza el desarrollo, se ha integrado **Zustand**. Este almacén inyecta la telemetría recibida por WebSockets directamente a los marcadores del mapa y a las gráficas, aislando los renderizados innecesarios.
- **Diseño Responsivo y PWA (Progressive Web App):** La interfaz ha sido maquetada utilizando CSS modular y *Media Queries* fluidas, garantizando que el cuadro de mandos sea perfectamente operable tanto en monitores de centro de control como en las *tablets* rugerizadas de los operarios a pie de campo. Adicionalmente, el proyecto integra el conector `vite-plugin-pwa`, permitiendo que la web se instale como una aplicación nativa en dispositivos móviles y cachee sus recursos estáticos (*Service Workers*), una característica vital para operar en parcelas agrícolas con conectividad intermitente.
- **Analítica de Datos (Recharts):** La visualización de históricos de carga energética y avance de misión se delega en la librería vectorial `recharts`, que procesa matrices masivas de datos para dibujar gráficos de líneas interactivos en SVG.
- **Exportación Documental e Internacionalización:** Dado que la plataforma puede operar en diversos entornos, incorpora `i18next` para abstraer las cadenas de texto y permitir el cambio de idioma en caliente sin recargar. Adicionalmente, el ecosistema integra `html2canvas` junto con `jsPDF` para vectorizar las gráficas de rendimiento y ofrecer a los responsables la capacidad de exportar informes físicos en formato PDF con un solo clic.

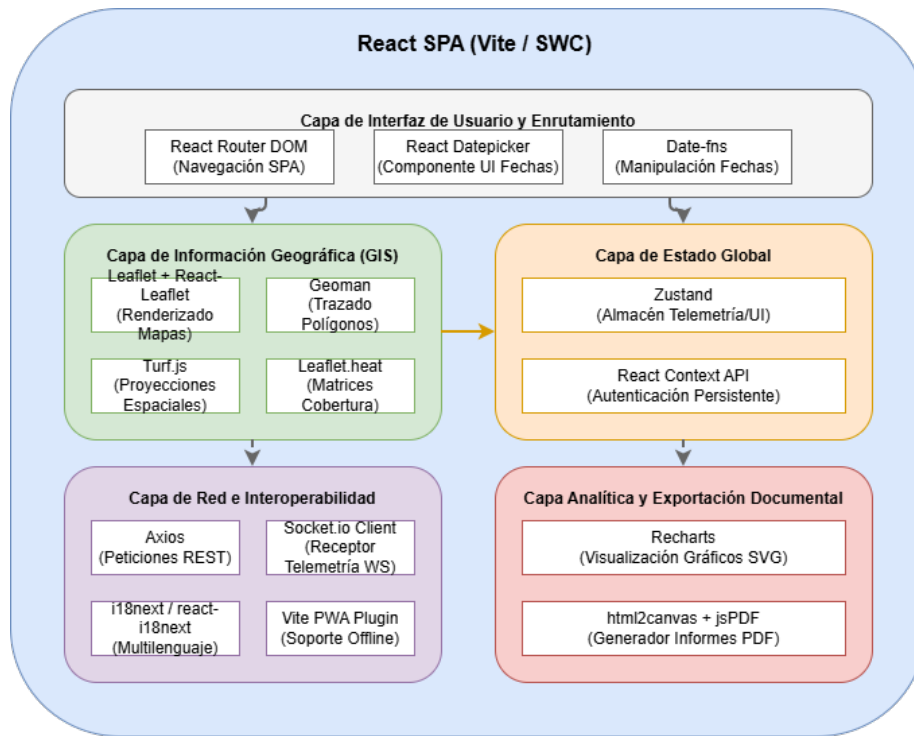


Figura C.10: Diagrama de Arquitectura Interna del Frontend (Ecosistema React)

Modelo de Seguridad, Resiliencia y Mitigación de Ataques

Dado que el *software* emula un entorno agro-industrial crítico, la arquitectura ha sido reforzada a nivel *DevOps* incorporando mitigadores directos frente a ataques cibernéticos modernos y fallos en tiempo de ejecución:

1. **Identidad y Control de Acceso (RBAC):** El control de autorización se implementa mediante el modelo *Role-Based Access Control* (Administrador, Operario, Visualizador). El estado de autenticación no se guarda en memoria del servidor (*stateless*), sino que fluye encapsulado y firmado criptográficamente en *JSON Web Tokens* (JWT), asegurando escalabilidad infinita si se balancea la carga. Las contraseñas están ofuscadas en base de datos mediante el algoritmo de coste adaptativo *bcrypt*.
2. **Prevención de DDoS (Denegación de Servicio):** Las infraestructuras conectadas a la red corren el riesgo de ser saturadas por

peticiones masivas. Se ha integrado la librería `express-rate-limit` a nivel de enrutador raíz, la cual estrangula proactivamente el tráfico si una misma IP excede el umbral seguro de 200 peticiones en 15 minutos.

3. **Prevención de Memory Exhaustion:** La ingestión de polígonos y datos geospaciales es un vector de ataque clásico para desbordar la memoria RAM. El sistema está acorazado explícitamente mediante el interceptor `express.json({ limit: "1mb" })`, descartando instantáneamente cargas maliciosas o excesivamente pesadas antes de que sean procesadas por la lógica de dominio.
4. **Cabeceras Seguras y Geofencing Lógico:** El módulo `helmet` reescribe activamente las respuestas HTTP para prevenir ataques XSS (Cross-Site Scripting) mediante la inyección de *Content-Security-Policies* (CSP). Adicionalmente, dado que un robot agrícola suele operar en redes LAN aisladas, el cortafuegos lógico CORS rechaza taxativamente el tráfico que no proceda del *localhost* o de subredes autorizadas (ej. `192.168.x.x`).
5. **Resiliencia de Subprocesos (Crash Prevention):** Node.js tiene un comportamiento determinista de colapso frente a promesas huérfanas. Para evitar que una anomalía matemática del simulador apague el servidor entero, el archivo principal implementa escuchadores activos (`uncaughtException` y `unhandledRejection`) que atrapan la excepción en memoria, imprimen una traza en el log y degradan el servicio de manera elegante sin corromper el estado de la máquina física real.

Arquitectura de Pruebas y Control de Calidad (QA)

Para asegurar la robustez de un sistema con lógicas tan pesadas como el *pathfinding* geográfico y la simulación matemática, se ha implantado una infraestructura de pruebas de grado industrial.

En lugar de basar la fiabilidad en la simple comprobación manual del desarrollador, el proyecto incluye *suites* de pruebas automatizadas:

- **Frontend (Vitest):** Se utiliza Vitest, el *framework* de pruebas nativo de Vite, para aislar y validar el comportamiento de los componentes React y funciones puras de utilidad de manera ultrarrápida en entornos de terminal.

- **Backend (Jest y Supertest):** La lógica de servidor cuenta con pruebas asiladas gobernadas por Jest. La librería **supertest** actúa como un agente HTTP falso que bombardea el API REST y valida los códigos de estado devueltos (200 OK, 403 Forbidden, 400 Bad Request) y la estandarización JSON de las respuestas.
- **Pruebas End-to-End (Testcontainers):** Probablemente la tecnología más avanzada de la fase QA. En los *tests* de integración profundos (*E2E*), el sistema utiliza **@testcontainers/postgresql**. Esto significa que, durante la ejecución del *test*, Node.js se comunica con el demonio de Docker del ordenador para levantar dinámicamente un contenedor efímero real de PostgreSQL, inyectar datos de prueba, realizar las afirmaciones (*assertions*) pertinentes, y destruir el contenedor al finalizar. De este modo, se verifica la interacción real entre el ORM Prisma y el motor relacional sin riesgo de contaminar la base de datos de producción.

Apéndice *D*

Documentación técnica de programación

D.1. Introducción

Este apéndice ofrece el contexto técnico necesario para comprender la implementación a bajo nivel del *software* AgroSkopos, sirviendo como manual de referencia (*onboarding*) para futuros desarrolladores o equipos de mantenimiento.

Se detalla la justificación organizativa de los archivos, los paradigmas de modularidad aplicados y los principios de separación de responsabilidades que permiten que el código sea escalable y resistente a la alta complejidad.

D.2. Estructura de directorios

La base de código fuente se organiza bajo el paradigma de **Monorepo** (repositorio monolítico). Este patrón arquitectónico centraliza múltiples proyectos lógicos (en este caso, el cliente web y el servidor) dentro del mismo control de versiones, facilitando la integración continua (CI/CD), la orquestación conjunta mediante Docker y la coherencia estructural.

Organización del repositorio raíz

En el directorio raíz del proyecto residen los archivos fundamentales de configuración y orquestación, de los cuales cuelga toda la lógica de la aplicación.

```

/
├── .github/ -> Flujos de integración continua (CI/CD)
├── app/ -> Código fuente dividido en cliente y servidor
│   ├── client/ -> Cliente web interactivo (React + Vite)
│   └── server/ -> Servidor API REST y motor de simulación (Node.js)
├── database/ -> Scripts de inicialización de la base de datos
├── documentacion_tfg/ -> Memoria académica y anexos en  $\LaTeX$ 
├── .env.example -> Plantilla estandarizada de variables de entorno
├── docker-compose.yml -> Definición de infraestructura Docker
├── package.json -> Scripts globales del monorepo y linter
└── sonar-project.properties -> Configuración de análisis estático (SonarCloud)

```

Estructura del cliente web (Frontend)

El cliente de React abandona la tradicional agrupación plana por tipos técnicos (donde todas las vistas estarían en una sola carpeta enorme) en favor de un modelo híbrido orientado al dominio y a características (*Feature-Sliced Design*). Esta decisión semántica garantiza que cada módulo de la aplicación (por ejemplo, el panel de control o la autenticación) actúe de forma encapsulada y autocontenida.

```

app/client/
├── src/ -> Código fuente principal de React
│   ├── components/ -> Componentes UI "tontos" genéricos (botones, modales)
│   ├── config/ -> Variables estáticas y parámetros de despliegue
│   ├── context/ -> Proveedores globales de estado estático (ej. Autenticación)
│   ├── features/ -> Módulos complejos de dominio encapsulados
│   │   ├── authentication/ -> Lógica y vistas de inicio de sesión o registro
│   │   ├── control/ -> Panel e interfaces de operación del robot
│   │   └── dashboard/ -> Vistas de resumen, analíticas y métricas
│   ├── hooks/ -> Lógica reactiva y ciclos de vida (Custom Hooks)
│   ├── i18n/ -> Ficheros de traducción e internacionalización
│   ├── layout/ -> Estructuras base de la interfaz (Navbar, Sidebar)
│   ├── pages/ -> Ensambladores de alto nivel o vistas completas
│   ├── store/ -> Estado global masivo de alta frecuencia (Zustand)
│   ├── App.jsx -> Enrutador raíz y definición de accesos
│   └── main.jsx -> Punto de entrada e inyección de la aplicación en el DOM
├── public/ -> Activos estáticos directos (manifiesto PWA, favicon, iconos)
│   ├── avatars/ -> Imágenes de perfil para los usuarios
│   └── pwa-icons/ -> Iconos generados para la aplicación instalable PWA
├── package.json -> Dependencias, scripts de test y comandos de Vite
└── vite.config.js -> Compilador SWC y configuración de empaquetado

```


Estructura del servidor (Backend)

El motor del sistema (*backend*) en Express.js sigue una separación estricta de responsabilidades (Capa de Presentación, Capa de Lógica de Negocio y Capa de Acceso a Datos), garantizando que los flujos de información sean fuertemente tipados y predecibles.

```
app/server/
├── src/ -> Código fuente del motor en Node.js
│   ├── config/ -> Inicializadores (Variables de entorno, Swagger)
│   ├── controllers/ -> Receptores HTTP y gestión de respuestas REST
│   ├── generated/ -> Clientes dinámicos o auto-generados
│   ├── middlewares/ -> Interceptores de tráfico de red (Seguridad, Errores)
│   ├── routes/ -> Enrutador y asignación semántica de los endpoints
│   ├── schemas/ -> Reglas de validación de datos estrictas mediante Zod
│   ├── scripts/ -> Tareas administrativas (ej. poblar la base de datos)
│   ├── services/ -> Lógica pura de dominio e interacciones ORM (Prisma)
│   ├── websockets/ -> Gestores de canal bidireccional en tiempo real
│   ├── index.js -> Inicialización del puerto web y servicios pasivos
│   └── simulator.js -> Motor activo (cálculos físicos, geográficos y batería)
├── prisma/ -> Modelo relacional de base de datos (schema.prisma)
└── package.json -> Dependencias y comandos de ejecución de Node.js
```

Arquitectura de control de calidad (Colocation Principle)

Un elemento clave en la arquitectura técnica de AgroSkopos es la estructuración organizativa de los archivos de pruebas. De forma deliberada, el código rompe con el patrón clásico de almacenar todas las pruebas unitarias y funcionales en una inmensa carpeta global (como `tests/`) en la raíz.

En su lugar, el proyecto adopta firmemente el **Principio de Colocation**: tanto en el servidor como en el cliente web, las carpetas `__tests__` o los archivos sufijados en `.test.js` se declaran **exactamente en el mismo directorio** que el código fuente que evalúan (ej. `DataPage.test.jsx` vive en el mismo directorio que `DataPage.jsx`). Este enfoque presenta dos ventajas vitales de ingeniería:

1. **Mantenibilidad e Intención:** Un desarrollador localiza al instante la cobertura de pruebas de un archivo sin navegar por árboles inconexos, lo que fomenta de forma natural el *Test-Driven Development* (TDD).

2. **Refactorización resiliente:** Al mover un componente React o un servicio de Node.js a otro directorio para reestructurar el sistema, su test viaja encapsulado automáticamente con él, impidiendo que se rompan las frágiles rutas relativas de importación (`../..../`).

La única excepción a este principio se aplica a las **pruebas E2E (End-to-End)**. Dado que estas validan el sistema web en su conjunto interactuando con toda la pila tecnológica (levantando bases de datos de prueba en la subred mediante *Testcontainers*), dichas pruebas sí residen en infraestructuras separadas debido a su naturaleza global y no unitaria.

D.3. Manual del programador

Esta sección tiene como propósito servir de guía técnica para futuros desarrolladores que necesiten mantener, extender o depurar el sistema AgroS-kopos. Se documentan aquí los patrones de arquitectura aplicados, el ciclo de vida de los datos y las convenciones de programación adoptadas, de manera complementaria a los diagramas arquitectónicos expuestos en el Anexo C.

Calidad estática del código

El repositorio emplea herramientas automatizadas para garantizar un estilo de código homogéneo y libre de defectos lógicos. En los proyectos de cliente y servidor se utilizan **ESLint** para la validación estática. Además, todo el código subido al repositorio es analizado de forma continua por **SonarCloud** (según lo descrito en la configuración CI de *GitHub Actions*), que mide la deuda técnica, las vulnerabilidades y la cobertura de pruebas. Cualquier nuevo desarrollo debe adherirse a estas normativas para superar las barreras de integración. Adicionalmente, es un requisito estricto mantener y ampliar la batería de pruebas (unitarias y E2E) a medida que se escribe nuevo código o se modifican funcionalidades existentes, asegurando que la cobertura no disminuya y protegiendo el sistema continuamente contra regresiones.

El flujo de una petición en el Backend (API REST)

El servidor, construido sobre Node.js y Express, implementa una filosofía de validación rígida y encapsulación lógica. El flujo típico cuando se recibe una solicitud externa (por ejemplo, para crear una misión) es el siguiente:

1. **Capa de Rutas (`routes/`):** Recibe la petición HTTP y la delega a los interceptores correspondientes.

2. **Capa de Validación Estricta (`schemas/`):** Antes de invocar a la lógica de negocio, se validan los parámetros de entrada utilizando la librería **Zod** (ej. `missionSchema.js`, `robotSchema.js`). Si la estructura o los tipos de datos no coinciden con los esperados, la solicitud es interceptada y rechazada automáticamente, impidiendo posibles inyecciones o corrupciones en la base de datos.

3. **Capa de Controlador (`controllers/`):** Extrae la información ya formateada y limpia, para pasársela a los servicios.

4. **Capa de Servicios (`services/`):** Ejecuta la lógica empresarial pura y realiza las transacciones necesarias empleando **Prisma ORM** contra la base de datos PostgreSQL.

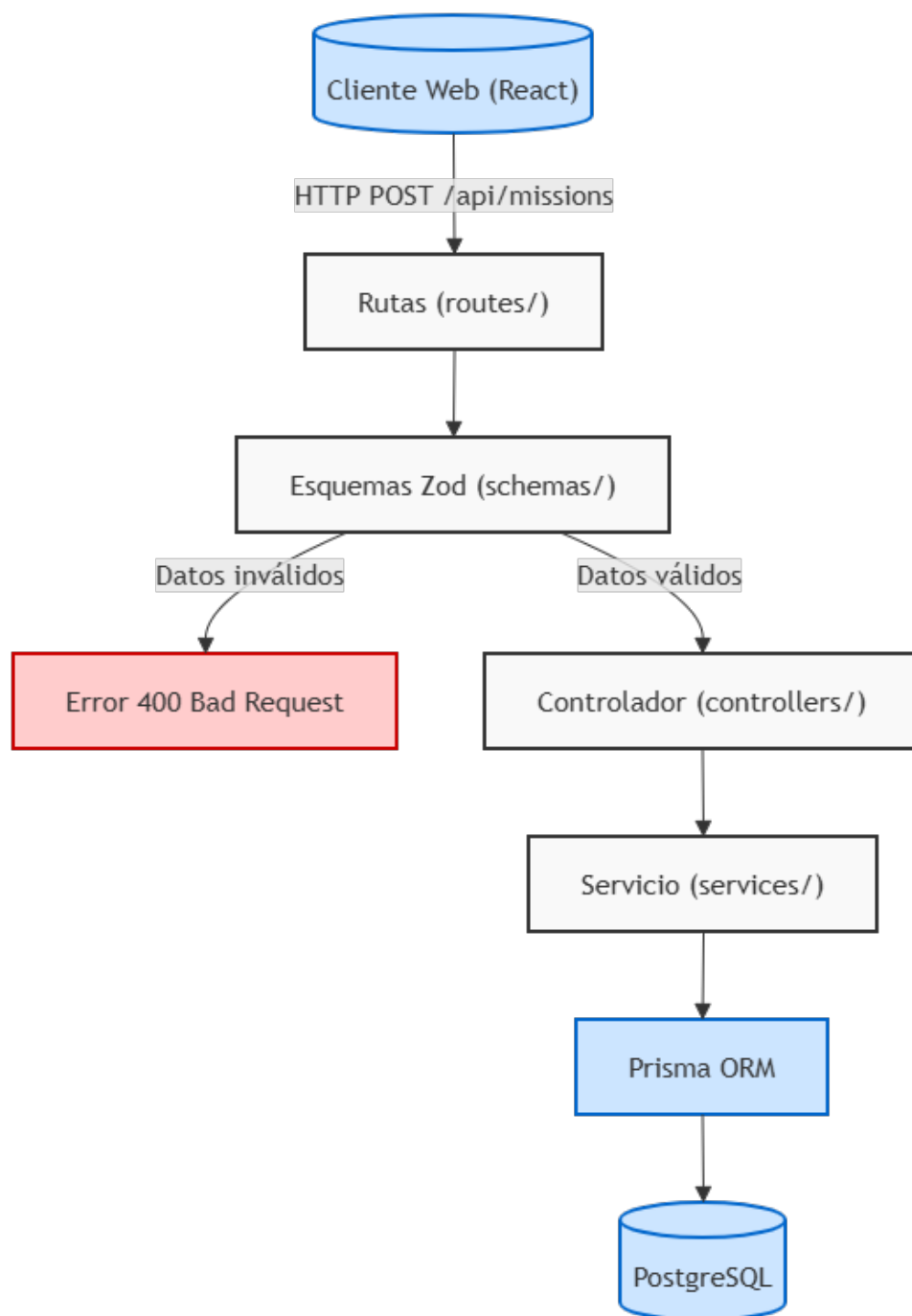


Figura D.1: Diagrama de flujo del ciclo de vida de una petición HTTP en la API REST.

Manejo de Errores

Actualmente, los errores asíncronos y fallos de negocio se gestionan atrapando las excepciones en la capa de controladores y devolviendo respuestas JSON estandarizadas con códigos de estado HTTP semánticos (400, 401, 404, 500) acompañados de un mensaje descriptivo para la correcta interpretación del *frontend*.

Comunicaciones asíncronas y simulación física

Uno de los componentes centrales y más complejos del servidor es el **Motor de Simulación** (`simulator.js`). Este módulo emula en tiempo real la física y las operaciones del robot agrícola, incluyendo:

- Generación de rutas de cobertura y barrido óptimas usando mallas de polígonos geográficos (`@turf/turf`).
- Consumo dinámico de batería basado en la velocidad lineal, y recargas mediante simulación de eficiencia de paneles solares.
- Control de estados autónomos: ciclos de inactividad (*IDLE*), reanudación de misiones y retorno seguro a la base (*RTL*).

Debido a su naturaleza en tiempo real, los datos calculados por el simulador no viajan mediante protocolo HTTP tradicional. Se emplea un canal bidireccional asíncrono con **WebSockets** (`websockets/socketHandler.js`), el cual emite paquetes de telemetría constantes (*ticks* de velocidad, latitud, longitud, nivel de batería) a todos los clientes web conectados simultáneamente.

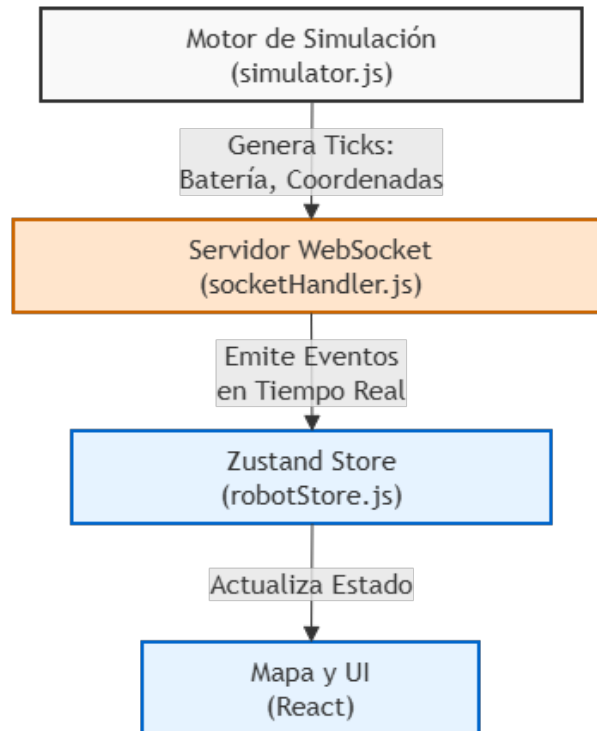


Figura D.2: Diagrama del flujo asíncrono de datos emitidos por el simulador mediante WebSockets.

Gestión de estado y arquitectura en el Frontend

Para evitar bloqueos y cuellos de botella en la interfaz interactiva (React), AgroSkopos aborda la gestión de estado de forma segregada:

- **Estado Global de alta frecuencia (Zustand):** Para suscribirse a la ráfaga de eventos del WebSocket (como las coordenadas en vivo del robot) y gestionar entidades complejas, se utiliza *Zustand*. Este se organiza de manera modular en el directorio `store/` (ej. `robotStore.js`, `missionStore.js`), permitiendo actualizaciones del DOM ultrarrápidas y sin renderizados innecesarios.
- **Estado Local y Contextos:** Se reserva para datos más estáticos o globales por usuario, como la autenticación (`AuthContext`) y controles de interfaz visual puramente local.

Guía para extender la interfaz (Feature-Sliced Design)

Si un desarrollador necesita agregar un nuevo módulo interactivo, debe adherirse al paradigma arquitectónico aplicado en `app/client/src/features/`. Para añadir una funcionalidad (ej. Módulo de análisis de suelo), los pasos recomendados son:

1. **Lógica de dominio (`features/`):** Crear una nueva carpeta bajo `features/soil_analysis/` para aislar sus componentes React, llamadas a la API y *hooks* exclusivos.
2. **Elementos visuales genéricos (`components/`):** No reinventar la rueda; utilizar los botones, modales o tarjetas genéricas ya existentes en la librería compartida `components/`.
3. **Estado Global (`store/`):** Si el módulo necesita leer o escribir variables que deben verse en toda la aplicación (ej. datos del suelo que también usa el dashboard), crear su propio *store* de Zustand.
4. **Ensamblaje y Rutas (`pages/` y `App.jsx`):** Exponer desde `features/` únicamente el bloque principal y usarlo para construir una página en `pages/SoilAnalysisPage.jsx`, registrándola finalmente en el enrutador `App.jsx`.

D.4. Compilación, instalación y ejecución del proyecto

D.5. Pruebas del sistema

El aseguramiento de la calidad del *software* (QA) en AgroSkopos se sustenta sobre un marco de pruebas automatizadas en múltiples niveles (unitarias y de extremo a extremo), abarcando tanto la lógica del servidor (*backend*) como las interfaces de usuario del cliente web (*frontend*).

Pruebas del Cliente Web (Frontend)

Para el entorno interactivo en React, se ha optado por el ecosistema de **Vitest** combinado con **React Testing Library** y `jest-dom`. Vitest proporciona un motor de ejecución ultrarrápido y nativo para el empaquetador Vite, eliminando las fricciones de configuración clásicas. Las pruebas se

centran en el comportamiento observable por el usuario en lugar de detalles de implementación interna. Se validan:

- **Componentes UI:** Renderizado correcto, manejo de eventos (*clicks*, *inputs*) utilizando `@testing-library/user-event` y flujos de renderizado (ej. `Modal.test.jsx`).
- **Lógica de estado y red:** Pruebas sobre contextos de React (ej. `AuthContext.test.jsx`) y utilidades cliente (ej. `httpClient.test.js`).

Los comandos disponibles definidos en el `package.json` del cliente incluyen:

- `npm run test`: Ejecuta la batería de pruebas mediante Vitest.
- `npm run test:cov`: Ejecuta las pruebas e instrumenta el código con el motor v8 para generar un informe completo de cobertura.

Pruebas del Servidor (Backend)

En el lado del servidor (Node.js con Express), se emplea **Jest** como marco principal de *testing*. El entorno se ha configurado para operar nativamente con módulos ES (`-experimental-vm-modules`). Se distinguen dos categorías de pruebas:

Pruebas unitarias

Siguen el *Principio de Colocation* detallado en la sección D.2.4. Evalúan de forma aislada componentes lógicos individuales como controladores, *middlewares* y esquemas de validación de datos, simulando (*mocking*) las dependencias externas y accesos a base de datos.

- Ejecución: `npm run test:unit`
- Cobertura: `npm run test:unit:cov`

Pruebas End-to-End (E2E)

Dado que el servidor interactúa intensivamente con una base de datos PostgreSQL, las pruebas E2E (alojadas en la carpeta `app/server/tests/`) validan los flujos HTTP y de lógica de negocio completos empleando **Supertest**.

Para garantizar un entorno de pruebas aséptico, determinista y realista, se hace uso de la herramienta **Testcontainers** ([@testcontainers/postgresql](https://github.com/testcontainers/testcontainers-postgresql)). Esta librería levanta dinámicamente un contenedor Docker de PostgreSQL efímero antes de la ejecución de las pruebas, aplica las migraciones y lo destruye al finalizar, garantizando que los tests se ejecuten sobre un motor de base de datos idéntico al de producción.

- Ejecución secuencial: `npm run test:e2e`

Cobertura de Código (Coverage)

La cobertura de código (*Code Coverage*) es una métrica clave en ingeniería de software empleada para certificar el porcentaje del código fuente que ha sido ejecutado y validado durante la batería de pruebas automáticas.

La ejecución de los comandos de cobertura generará reportes que se pueden visualizar de dos formas principales: mediante un resumen estructurado directamente en la consola/terminal, y a través de una página web interactiva. Estos reportes HTML se generan dentro de la carpeta **coverage** (tanto en el proyecto de **client** como de **server**), y al abrirlos en el navegador permiten navegar línea por línea a través de todos los archivos del código para ver qué flujos exactos no se han evaluado. Estos reportes desglosan porcentualmente:

- **Statements (Declaraciones):** Proporción total de sentencias ejecutadas.
- **Branches (Ramas):** Proporción de bifurcaciones lógicas evaluadas (`if/else`, `switch`).
- **Functions (Funciones):** Proporción de métodos y funciones invocadas.
- **Lines (Líneas):** Porcentaje absoluto de líneas de código cubiertas.

Para demostrar la fiabilidad técnica de la aplicación, se genera un reporte con los resultados globales del análisis de cobertura del sistema, lo que refleja un alto grado de mitigación frente a regresiones.

Automatización en Integración Continua (CI)

Adicionalmente a la ejecución manual, el proyecto cuenta con un flujo de trabajo de integración continua configurado mediante **GitHub Actions**. Tal y como se define en el archivo `.github/workflows/ci-sonarcloud.yml`, cada vez que se suben cambios (*push*) a la rama principal o se interactúa con un *Pull Request*, una máquina virtual instalada en la nube ejecutará de forma desatendida todas las baterías de pruebas del monorepo (frontend, backend unitarias y backend E2E). Si alguna prueba falla, la subida se marca como errónea. Finalmente, los archivos de cobertura generados son procesados y enviados automáticamente a la plataforma **SonarCloud** para su análisis de calidad estático.

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción a la Sostenibilidad

Este anexo presenta una reflexión detallada sobre cómo el sistema AgroSkopos responde a los desafíos actuales de la sostenibilidad. Siguiendo las directrices de la Conferencia de Rectores de las Universidades Españolas (CRUE), el Trabajo de Fin de Grado no solo debe valorarse técnicamente, sino evaluando su impacto real. La transición hacia un modelo agrícola automatizado en entornos corporativos (B2B) ofrece una oportunidad inmejorable para integrar la ética profesional y la responsabilidad social corporativa. A continuación, se desgranar las tres dimensiones fundamentales de la sostenibilidad aplicadas a esta plataforma.

F.2. Dimensión Medioambiental

El pilar medioambiental constituye el núcleo fundamental de AgroSkopos. Históricamente, la agricultura a gran escala ha dependido de la sobreexplotación de recursos naturales y del uso intensivo de maquinaria de combustión fósil.

En primer lugar, el sistema fomenta la **Agricultura de Precisión**. Al recolectar datos a través de sensores (humedad, pH y NPK) y representarlos mediante mapas de calor, se proporciona a la empresa agrícola la inteligencia necesaria para aplicar el riego y los agroquímicos de forma localizada. Esta precisión previene uno de los problemas ecológicos más graves del sector: la

escorrentía superficial de químicos que acaba contaminando los acuíferos subterráneos.

En segundo lugar, el proyecto impulsa la **Reducción de la Huella de Carbono**. El diseño de la plataforma está orientado a operar con vehículos 100 % eléctricos que reemplazan a los altamente contaminantes tractores diésel. La integración de la telemetría demuestra que es viable mantener el rendimiento agrícola operando con cero emisiones directas en la parcela de trabajo.

F.3. Dimensión Económica

La adopción de AgroSkopos por parte de empresas del sector primario supone una optimización sin precedentes de sus recursos operativos.

La adopción de la agricultura de precisión garantiza un impacto ecológico positivo y se traduce de forma inmediata en un ahorro de costes significativo. La reducción en la adquisición y el despliegue indiscriminado de insumos químicos (pesticidas y fertilizantes sintéticos) y agua de riego, permite a las empresas agrícolas aumentar sus márgenes de beneficio minimizando sistemáticamente el desperdicio.

Adicionalmente, la capacidad de automatizar misiones reduce enormemente la dependencia del trabajo manual para tareas repetitivas de exploración. Este factor permite a las organizaciones reestructurar sus equipos, derivando a los empleados hacia labores de mayor valor añadido: tareas de gestión analítica, toma de decisiones estratégicas y supervisión a través de la plataforma web.

F.4. Dimensión Social y Ética Profesional

La introducción de flotas de robots autónomos conlleva profundas implicaciones sociales. Lejos de suponer una amenaza para el empleo rural, sistemas como AgroSkopos actúan como un poderoso catalizador para la **modernización estructural y la mejora de las condiciones laborales**.

El trabajo agrícola tradicional exige un esfuerzo físico extremo y exposición continuada a factores de riesgo (insolación severa y manipulación de compuestos químicos tóxicos). Al delegar estas tareas en el vehículo autónomo, el perfil del trabajador humano asciende a un rol de operador tecnológico. De esta forma, se dignifica el trabajo de campo y se reducen

drásticamente las tasas de accidentes laborales y enfermedades crónicas derivadas de la toxicidad química.

Desde la ética profesional de la ingeniería de software, la arquitectura incorpora validación estricta, gestión de usuarios basada en roles (RBAC) y mecanismos de seguridad activa (paradas de emergencia remotas), garantizando que las operaciones cumplan los estándares exigentes de seguridad corporativa.

F.5. Alineación con los Objetivos de Desarrollo Sostenible (ODS)

El diseño conceptual del proyecto AgroSkopos se alinea de forma transparente con la Agenda 2030 de las Naciones Unidas, impactando en los siguientes Objetivos de Desarrollo Sostenible:

- **ODS 2 (Hambre Cero):** El *software* mejora sustancialmente la resiliencia y el rendimiento de los cultivos mediante una monitorización continua, ayudando a producir mayor cantidad de alimentos utilizando menor cantidad de recursos.
- **ODS 6 (Agua Limpia y Saneamiento):** Al ajustar con precisión las dosis de fertilizantes estrictamente necesarias, se previene directamente la filtración de nitratos dañinos, protegiendo las reservas hídricas locales.
- **ODS 7 (Energía Asequible y no Contaminante):** El uso de esta plataforma con flotas de robots eléctricos impulsa al sector primario a abandonar progresivamente los combustibles fósiles.
- **ODS 15 (Vida de Ecosistemas Terrestres):** La mitigación en el uso indiscriminado de pesticidas en grandes extensiones previene activamente la degradación del suelo arable, protegiendo la biodiversidad local.

Bibliografía
